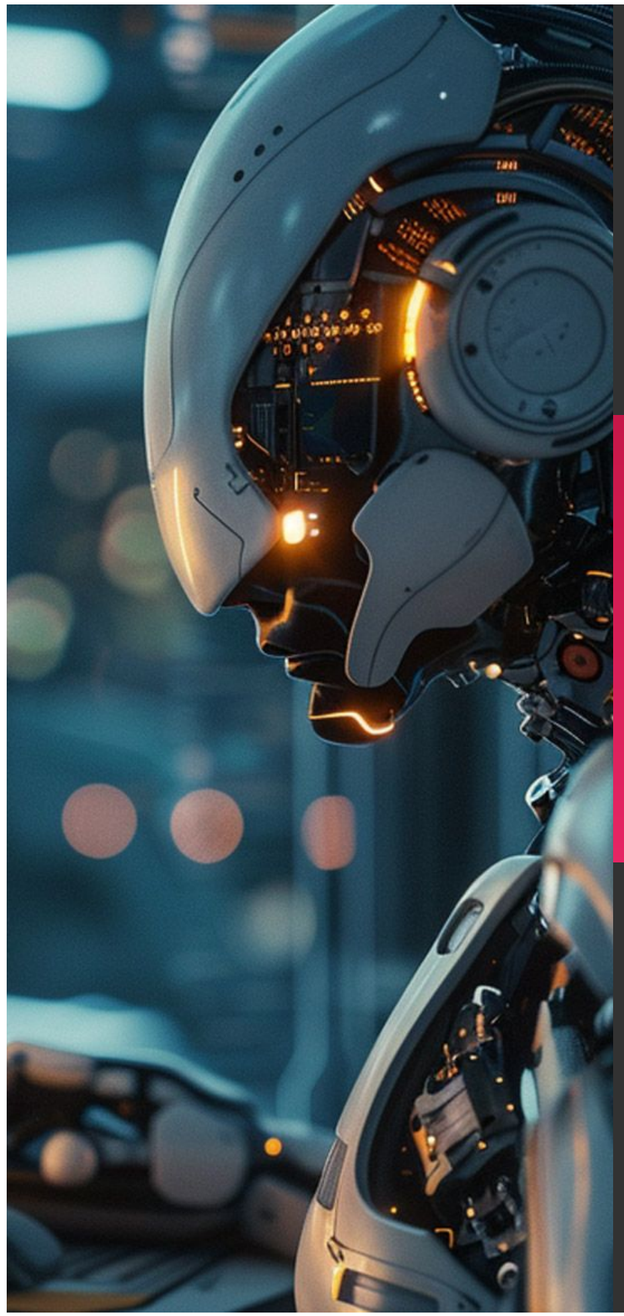


MLOps

The keys to mastering the production of AI models

Objectives

- Understand MLOps roles
- Understand the challenges of managing learning models
- Overview of the technologies used
- How to become an MLOps professional

The program

Key concepts

- MLOps
- MLFlow
- S3
- API
- CI/CD
- Docker / Docker Compose
- Kubernetes
- Monitoring
- Data Drift
- Retraining

https://ai-training.fr/

Summary of covered technologies

Objective : from POC (Proof Of Concepts) to model implementation **in production** 🚀

	Technology	Type	Objectives
	MLFlow	Model lifecycle management	Tracking and training reproducibility
	Amazon S3	Model registry	Store and versioning your models
	Flask, FastAPI, Bento ML	API	Make available model inference
	Docker	Containerization	Encapsulating / packaging your project
	Kubernetes	System distributed from containers	Scaling your applications
	Airflow	Orchestrator / scheduler	Create pipelines / workflows performance
	Prometheus and Grafana	Monitoring	Monitor pipelines and models



MLOPS

- **What is MLOps?**
- MLOps levels
- How to become a MLOps?

What is MLOps?

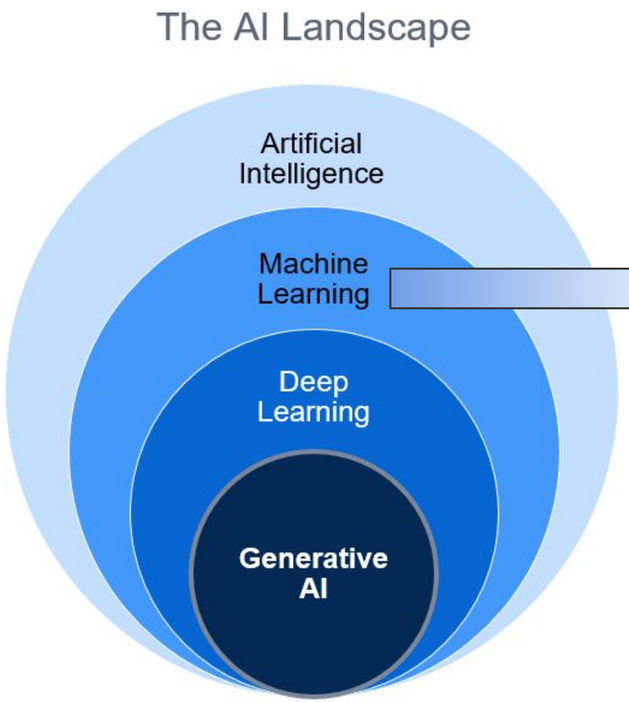


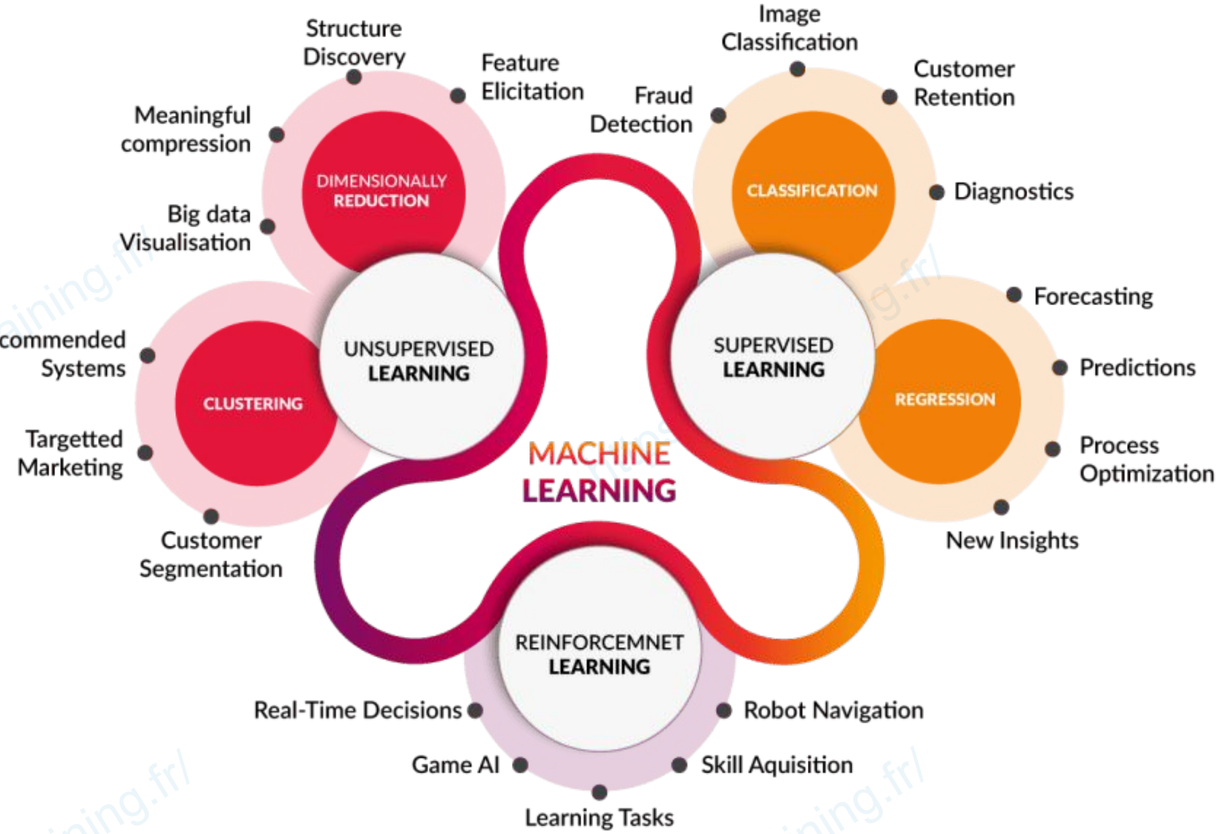
Introduction to the MLOps profession: role, objectives, scope and skills

Life cycle of a machine learning model

The challenges of managing learning models

A little background





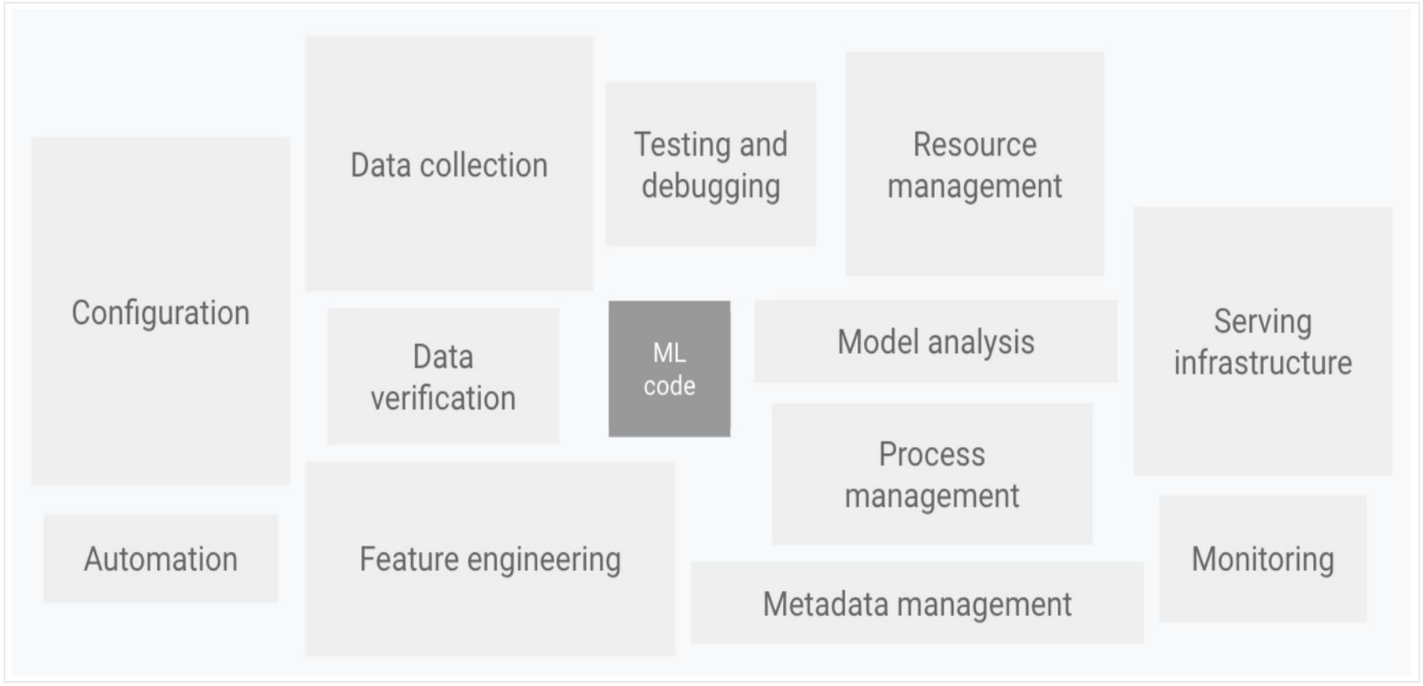
Source :
https://communities.sas.com/t5/SAS-Communities-Library/Where-does-GenAI-fit-within-the-AI-landscape/ta-p/915000

https://ai-training.fr/

ML's role in a project



Only a small part of a real ML system consists of ML code:



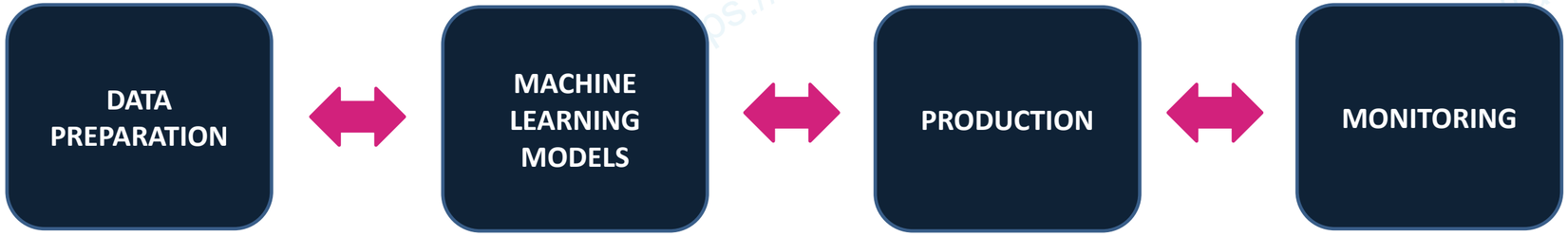
Source : https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

...the rest of the system encompasses:

- replayability of experiences
- configuration
- automation
- data collection
- data verification
- testing and debugging
- resource management
- model analysis
- process management
- metadata management
- deployment infrastructure
- monitoring
- retraining/rollback
- etc.

A little background

Machine learning models are playing an increasingly important role in business.



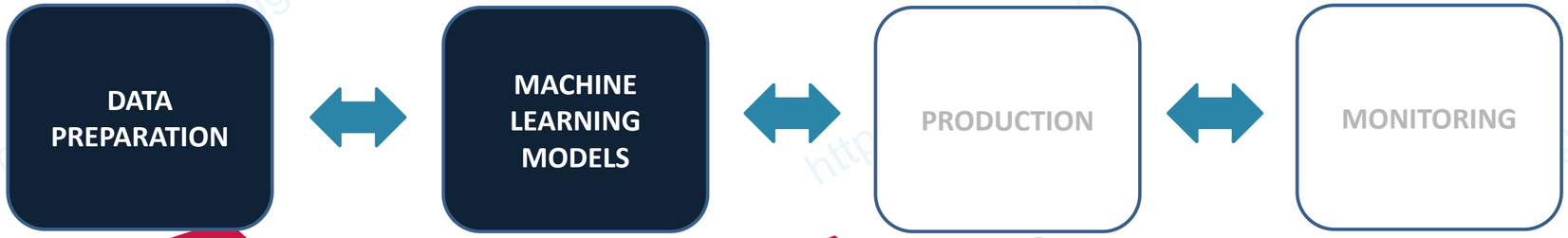
Data science teams face specific challenges that make it difficult to **put their work into production**.

https://ai-training.fr/

Key stages in a ML project

The process of delivering an ML model to production involves the following steps.
These steps can be performed **manually** or using an **automated pipeline**:

1. **Data extraction**: **selection** and **integration** of relevant data **from different sources** for the ML task.
2. **Exploratory data analysis** (EDA) to understand the **data schema**, the **features expected** by the model, identify **data preparation** and **feature extraction**
3. **Data preparation**: data **cleaning** and **transformation** (feature engineering), with creation of a training **set**, validation, and testing.
4. **Model training**: **benchmarking** of trained ML algorithms and **hyperparameter optimization**.
5. **Model evaluation**: the model is **evaluated** to determine its **performance** using **metrics**.

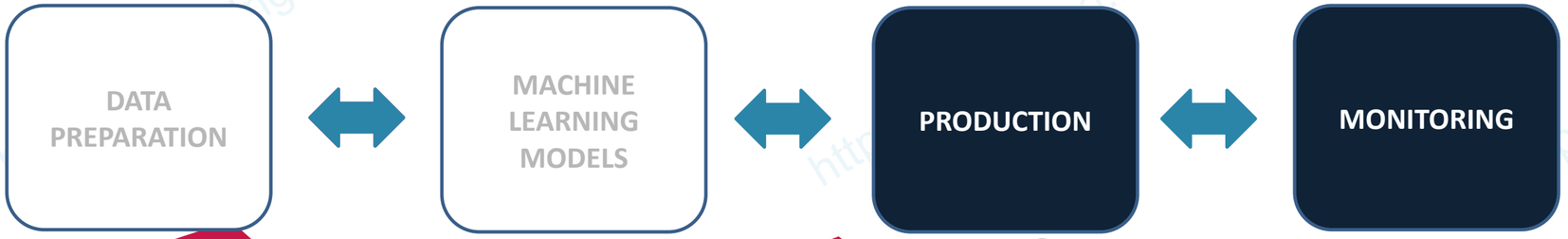


Key stages in a ML project

6. **Model validation**: the model is suitable for deployment if its predictive **performance exceeds** a certain business **threshold**.

7. **Model distribution** the validated model is **deployed in a target environment** to make predictions.
   This deployment can be one of the following:
   a. Microservices with **API REST** for online predictions
   b. Model integrated into an on-board or mobile system
   c. Part of a batch prediction system

8. **Model monitoring** the model's predictive performance is monitored with a view to calling a **new iteration** or **rollback** in the ML process.



https://ai-training.fr/

The challenges of managing learning models

Data science teams face **specific challenges** that make it **difficult** to put their work into production.

Reproducibility: training data, preparation, hyperparameters, storage, environment, etc.

Transparency and interpretability: predictions sometimes difficult to interpret

Scalability: adaptability to new data, new situations, etc.

Maintenance and monitoring: version management, performance monitoring, updates, etc.

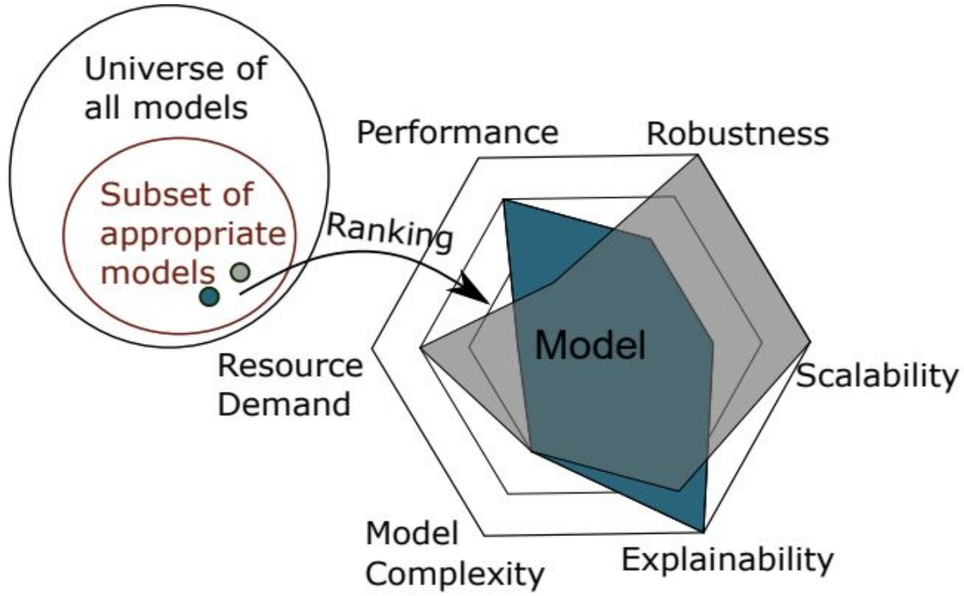
Governance: quality and conformity of data and model performance over time

Security: vulnerable to attack and bias in training games

https://ai-training.fr/

Model comparison

- **Performance:** model performance on new data
- **Robustness:** model resistance to inconsistent inputs and runtime environment failures
- **Scalability:** ability of the model to adapt to large volumes of data in production
- **Explainability:** direct or a posteriori explicability of the model
- **Model complexity:** the model capacity must be adapted to the complexity of the data
- **Resources:** model resource requirements for deployment and inference



Source :https://arxiv.org/pdf/2003.05155.pdf

https://ai-training.fr/

Special features of Machine Learning projects



Team skills: the team includes **data scientists/research scientist**.
These members may not be **experienced** software engineers capable of creating **production**.



Development ML is **experimental** by nature!
You need to test different features, algorithms, modeling, hyperparameters to find the best fit for the problem.
The challenge is to **follow** what worked and what didn't, and to guarantee the quality of the **reproducibility** while optimizing **reuse** of the code.



Tests in addition to **unit tests** and **integration** you need to **validate data**, **evaluate** the quality of the trained model, then **validate** this one.

https://ai-training.fr/

Special features of Machine Learning projects



Deployment: ML systems may require you to deploy a multi-step **pipeline** in order to **automatically retrain** and **deploy** the model.
This pipeline adds complexity and requires automating the steps that are performed manually by data scientists prior to deployment in order to train and validate new models.



Production: model performance may **decline** due to code that is not always optimal, but also due to **constantly changing data**.
You must therefore **monitor** your model's performance so that you can send it for **retraining** or perform a **rollback** when the values deviate from your expectations.

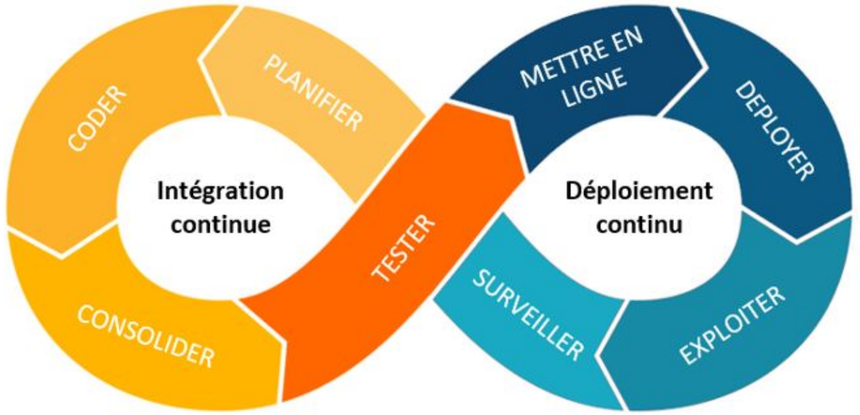
https://ai-training.fr/

CI, CD and CT

ML and other software systems are similar in terms of the **continuous integration** and the **continuous delivery** software module or package.

There are, however, a few **significant differences** ML side :

- **Continuous integration** (CI) is no longer just about testing and validating code and components, but also about **testing** and **validating data**, **data schemas**, and **models**.
- **Continuous delivery** (CD) no longer focuses on a single software package or service, but on a system (ML **training pipeline**) that must **automatically deploy** another service (model **serving service**).
- **Continuous training** (CT) is a **new feature** specific to ML systems, which involves automatically retraining and deploying models.



The goal is to **reduce** development cycles, increase deployment speed, and release reliable versions!

What is MLOps?



Introduction to the MLOps profession: role, objectives, scope and skills

The challenges of managing learning models

Life cycle of a machine learning model

Definition of MLOps

According to a study* conducted **by Gartner** in 2020, only **53 %** of AI projects move from the **from prototype to production**.

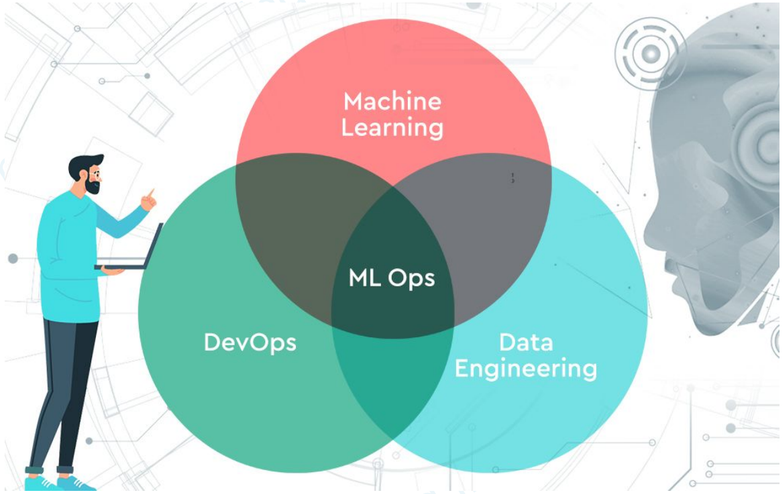
MLOps practices **palliate** this gap, and enable your teams to **accelerate** and **control** putting your AI models into production.



*Source :https://www.gartner.com/en/newsroom/press-releases/2020-10-19-gartner-identifies-the-top-strategic-technology-trends-for-2021

https://ai-training.fr/

Definition of MLOps



MLOps is the set of **practices** and **tools** which aim to :

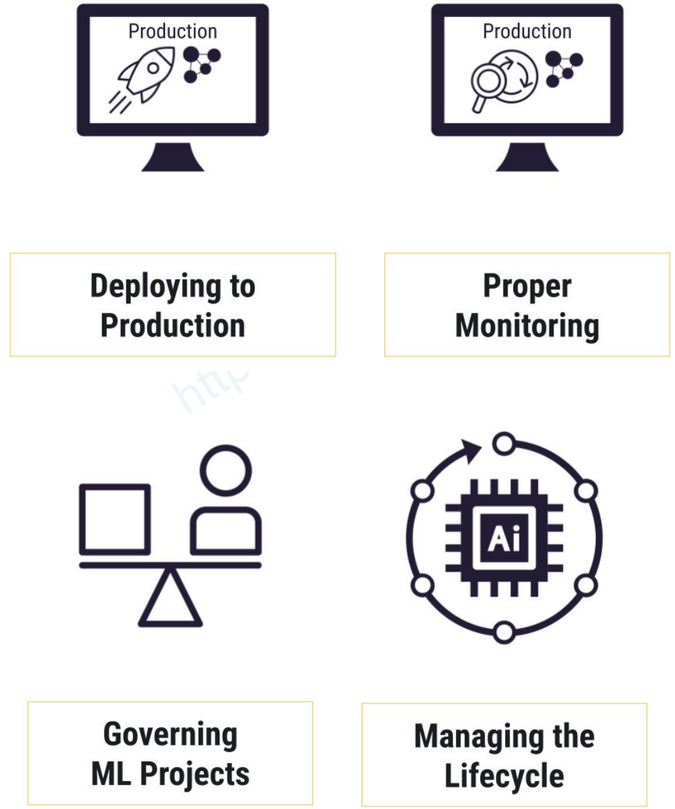
- **automate deployment processes**
- **manage** and **monitor** the whole **life cycle** machine-learning models, and their **development** to their **deployment** and their **maintenance**.

This approach addresses the specific challenges faced by data science teams and ensures that **production launch fast** and **efficient** ensuring ML **reliable, scalable,**
secure and **reproducible**.

https://ai-training.fr/

The roles of MLOps

- MLOps is responsible for managing **of the complete life cycle** ML models, from creation to production deployment
- MLOps is responsible for creating **pipelines** data for **collect**, **prepare** and **annotate** the data needed to train ML models
- MLOps is responsible for the management of **infrastructure** necessary for **train, deploy** and **monitor** ML models in production



Source :https://knowledge.dataiku.com/latest/mlops-o16n/architecture/concept-mlops-definition.html

https://ai-training.fr/

The roles of MLOps

- **Reduce the time needed** to train and deploy ML models in production
- **Improve** model **quality**, **tracking**, and **reliability** (through versioning and automated testing)
- **Reduce** development and maintenance **costs** by **automating** repetitive tasks (processing pipeline, retraining, deployment, etc.) and **improving error management**
- **Increase collaboration** and **communication** between development and ML teams

⇨ enables rapid and **efficient** production deployment, with **reliable**, **scalable**, **secure**, and **reproducible** models.

https://ai-training.fr/

The MLOps perimeter

- **Collection pipelines** of **preparation** of data and **training**
- Management **versions** of ML models
- **Tests automated** of ML models
- **Deployment** of ML models in production
- **Monitoring** of ML models in production
- **Maintenance** and **update** of ML models in production



https://ai-training.fr/

What is MLOps?



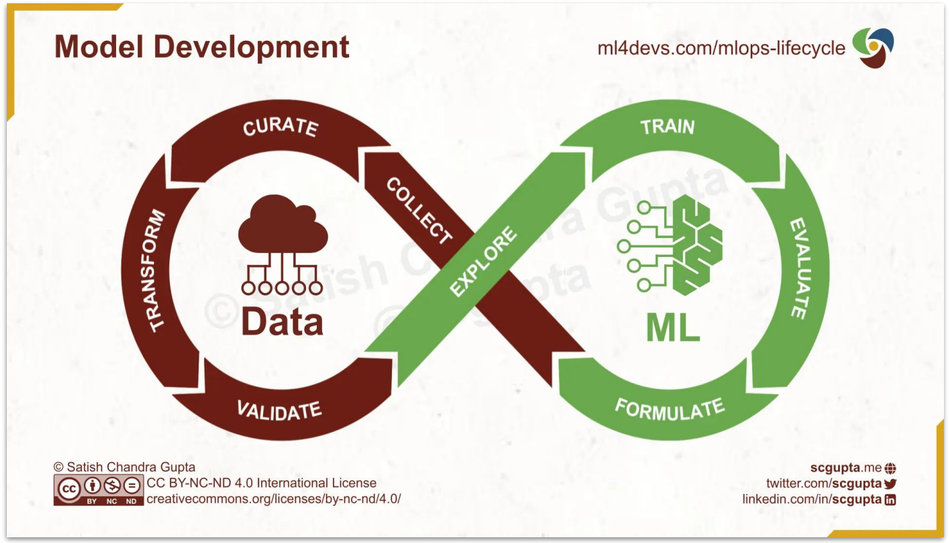
Introduction to the MLOps profession: role, objectives, scope and skills

Life cycle of a machine learning model

The challenges of managing learning models

Model development: Data + ML

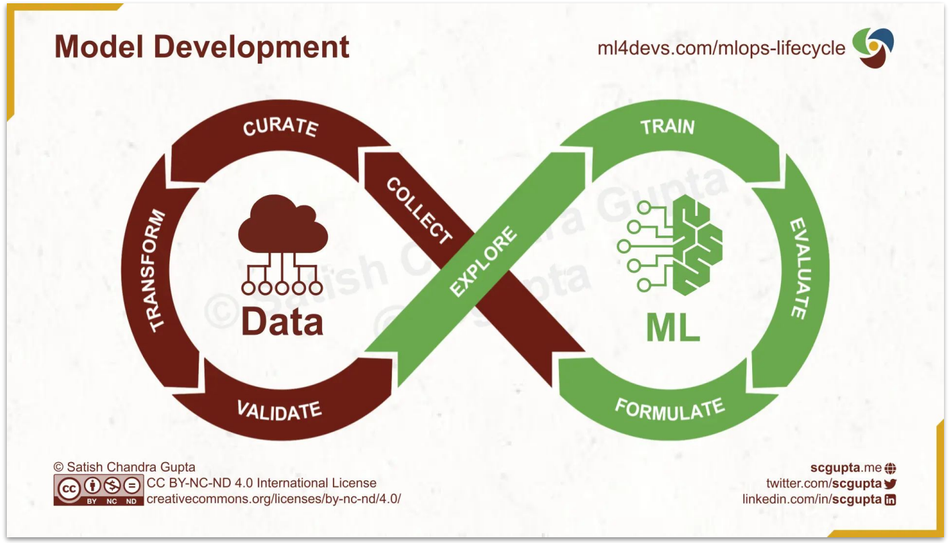
- **Formulate** definition of **problem** and **objectives** for the company
- **Collect**: **collect the data** (DB, Web scrapping, APIs, logs, etc.), integrated through **data pipelines**.
- **Curate** cleaning, deleting duplicates, filling in missing values, labeling, etc. Try to **automate as much as possible** but some parts will have to be done manually (labeling, for example)
- **Transform** transform data to make it suitable for ML analysis and modeling (join, **features engineering**, etc.)



https://ai-training.fr/

Model development: Data + ML

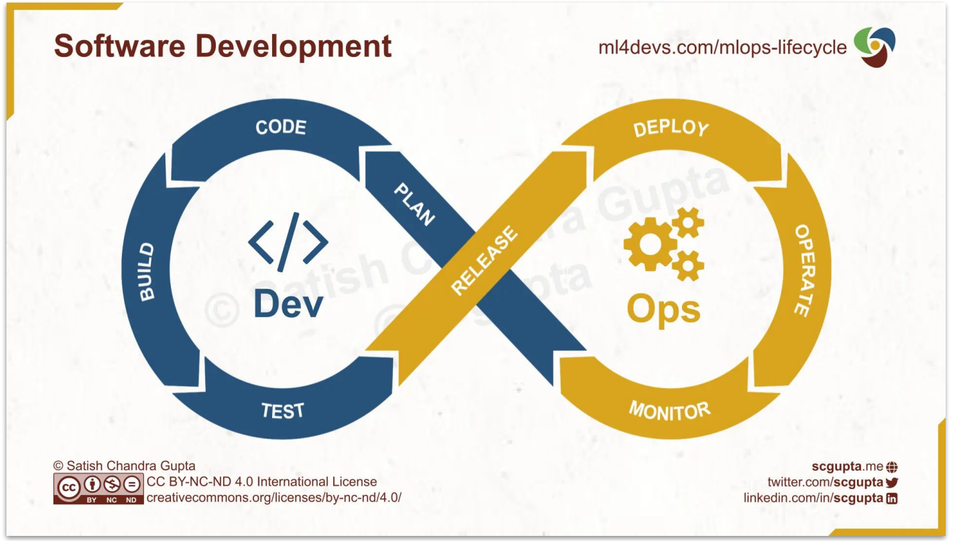
- **Validate**: validate and implement **data quality checks** maintain logs of statistical distributions over time and create alerts
- **Explore**: exploration for **descriptive statistics** the **exploratory analysis** or the **modeling**.
- **Train**: **Training** models, adjusting the **hyperparameters** and **model selection** the best performers
- **Evaluate**: Evaluate the characteristics and **performances model** in relation to the company's objectives. Certain feedbacks can lead to the ML problem being modified and formulated differently, and to **repeat** the whole process.



https://ai-training.fr/

Software development: Dev + Ops

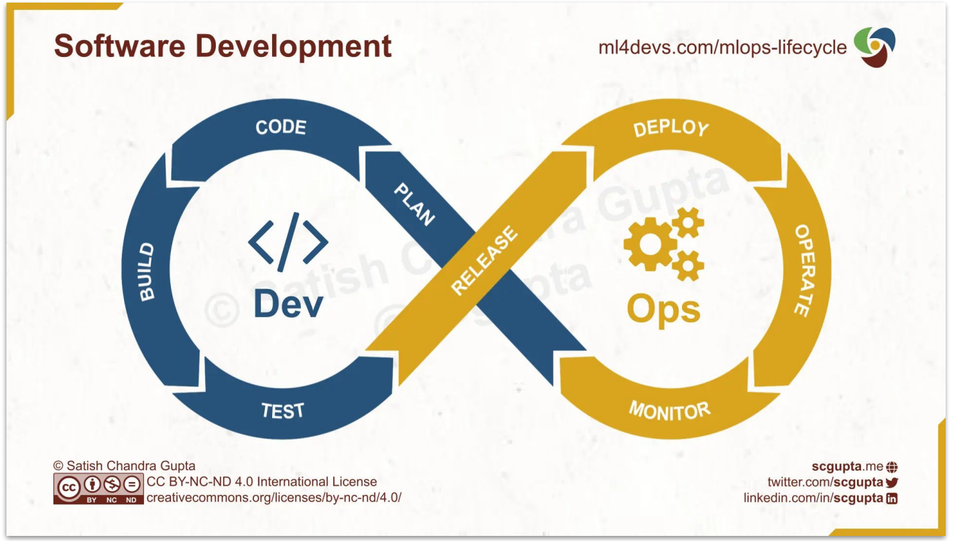
- **Plan**: discuss **objectives** to achieve and **required data** to evaluate **performances** of an ML model in the real world.
- **Code**: design and develop the software, product or application **from start to finish** (**API, Webapp, pipeline, etc.**) to get the teams on the same wavelength.
- **Build**: feeds the **continuous integration** various developments and their availability (libraries, SDK, Docker Image, binary, etc.)
- **Test** unit testing, functional testing, integration testing, performance testing, end-to-end testing, etc.
  automate them as much as possible.



https://ai-training.fr/

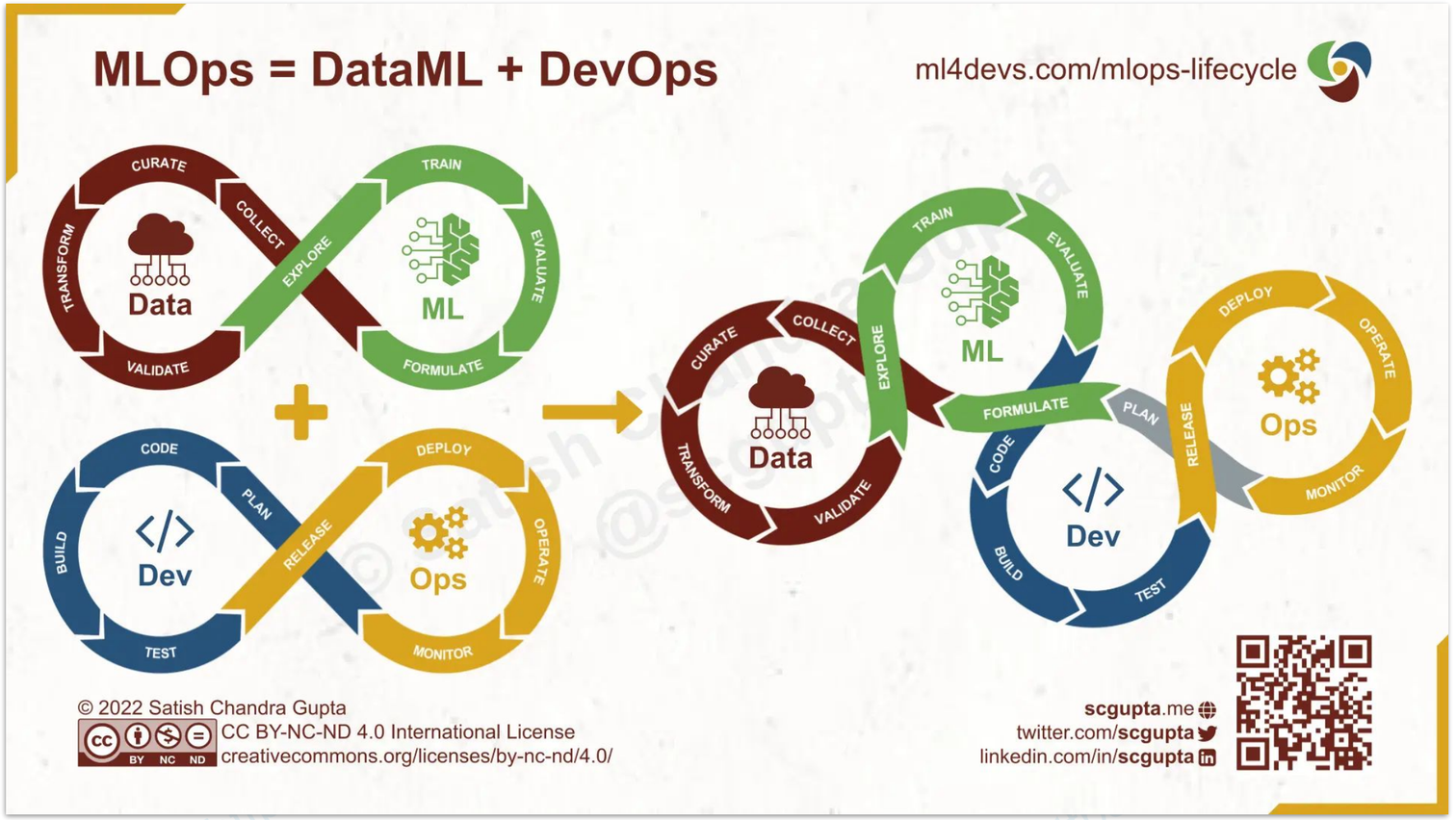
Software development: Dev + Ops

- **Release:** if the tests pass, the artefacts are made available: the **code**, the **models** and its **metadata** must be **versioned** and **saved** automatically.
- **Deploy:** choose the **artifacts** from the docker repo or the **model registry** and **deploy on infrastructure** production (via AWS SageMaker, PyTorch Serve, etc.)
- **Operate:** you can choose your own **deployment strategy** (blue-green deployment, canary, rollback, etc.). Kubernetes and KubeFlow are commonly used tools.
- **Monitor**: **monitor** continuously monitor service status, errors, latencies, model predictions, outliers, etc.
  Rollback or **re-training** if required



https://ai-training.fr/

MLOps = Data ML + DevOps



Three principles to improve the success rate of ML project development and deployment:

- **Versatility:** cross-functional team responsible for the entire project
- **Baseline**: Implementing a **simple model** (based on business or dummy rules) and test it from start to finish.
- **Iteration**: Build better models and replace the simple model, check and repeat



MLOPS

- What is MLOps?
- **MLOps levels**
- How to become MLOps?

https://ai-training.fr/

The levels of MLOps



The three levels of MLOps according to Google

0 manual process

1 pipeline ML

2 pipeline CI/CD

... and according to Microsoft

The three levels of MLOps

Google defines three levels* of MLOps

The level of automation of these steps defines the **maturity of the ML process**.

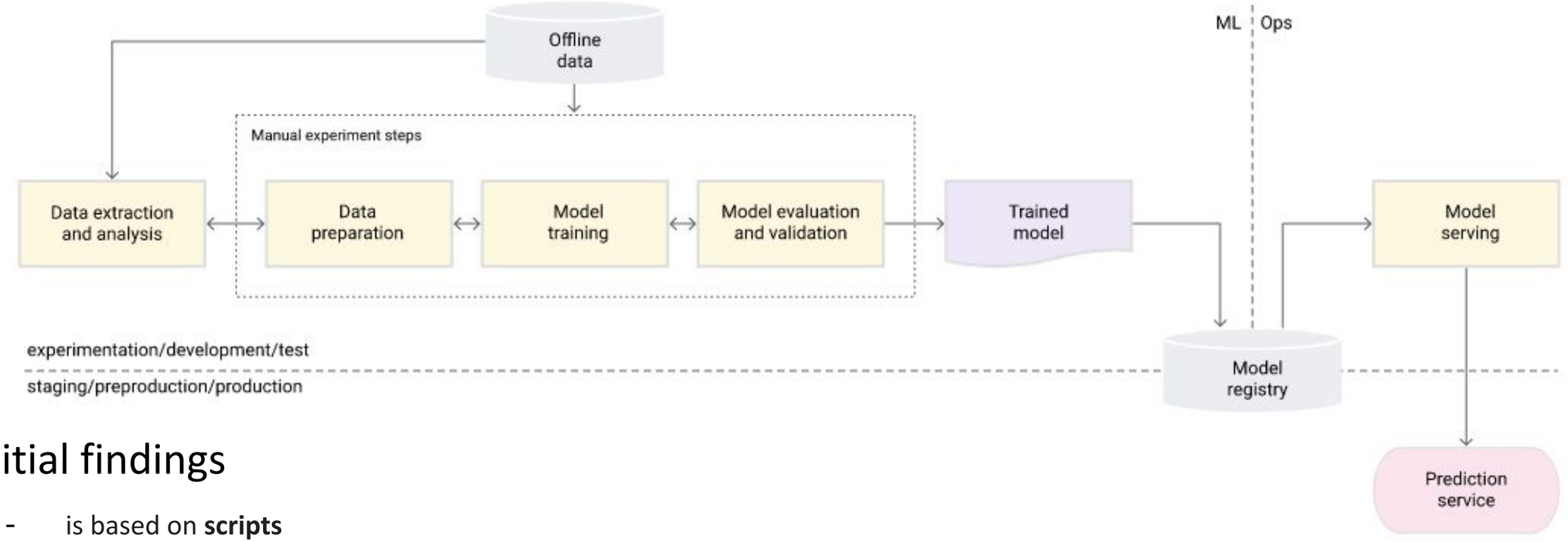
The following sections describe three levels of MLOps, from the most common, which involves no automation at all, to full automation of ML and CI/CD pipelines:

- Level 0: manual process
- Level 1: automation of the Machine Learning pipeline
- Level 2: CI / CD pipeline automation

https://ai-training.fr/

* Source : https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning?hl=en

Level 0 : manual process



Initial findings

- is based on **scripts**
- each step is **manual**
- **perf deviation** training / inference (because prod data)
- little or no re-training
- No CI/CD
- No monitoring
- Canary deployment (with or without AB testing)

Level 0 : manual process

Level 0 MLOps is **commonly used** by many companies who are starting to apply ML to their use cases.

This manual process, **driven by data scientists** can be **sufficient** when models are rarely modified or trained.

In practice, however, the models often have different **failures** when deployed in real-life conditions.

They **are unable to adapt to change** environment or data.

See: https://www.forbes.com/sites/forbestechcouncil/2019/04/03/why-machine-learning-models-crash-and-burn-in-production/

https://ai-training.fr/

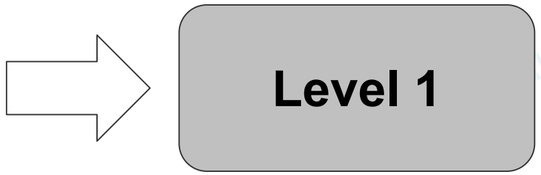
Level 0 : manual process

To meet these challenges :

- **Watch** the quality of your model in production: monitoring enables you to detect **performance degradations** and model obsolescence.
  It serves as a signal to launch a **new iteration** of tests and (manual) **retraining** of the model on new data.

- **Re-train** your models **frequently** in production: to capture ever-changing **trends**, you need to re-train your model with the **latest** data.

To meet the challenges of this **manual process** CI/CD and CT MLOps are useful.
By deploying a **ML drive pipeline** you can make **continuous training** and configure a **CI/CD system** to rapidly test, create and deploy new ML pipeline implementations.



https://ai-training.fr/

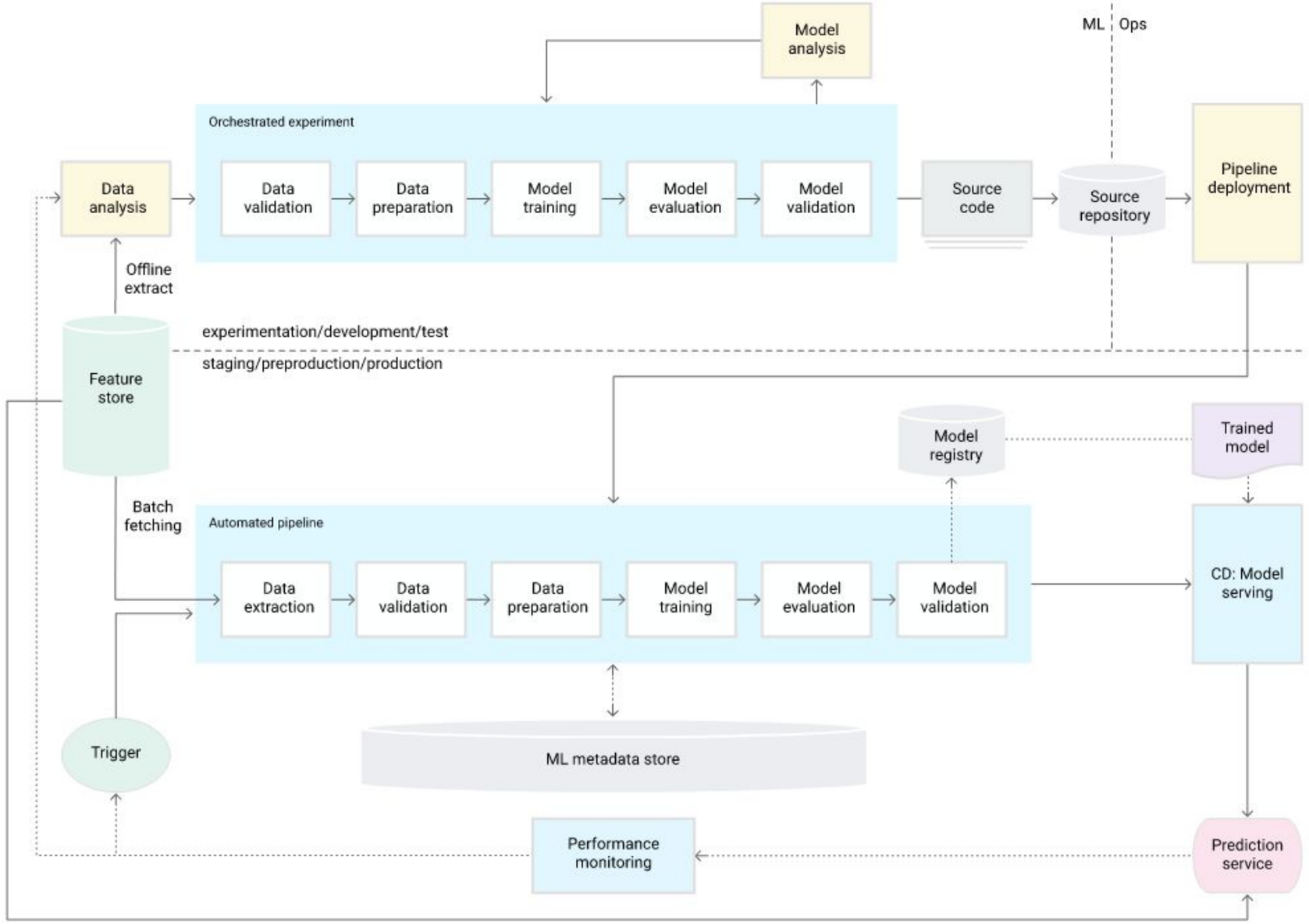
Level 1 ML pipeline automation

Objective

Continuous training model in **automating the ML pipeline**.
ML pipeline in production **continuously supplies** prediction services to new models trained on new data.

How?

By integrating **automated steps** validation of data and models, as well as **triggers** pipeline and a **metadata management**.



Level 1 ML pipeline automation

- **Quick test** ML test steps are **orchestrated**. The **transition** between stages is **automated** this enables rapid iteration of tests and a better launch into production.
- **Continuous training of production models** the model is **automatically driven in production** with new data on the basis of **triggers live pipeline**.
- **Experimental and operational symmetry** development or test environment, pre-production or production environment **have the same pipelines**
- **Modular code for components and pipelines** to create ML pipelines, components must be **reusable**, **composable** and potentially **shareable** between ML pipelines. Ideally, the components should be **containerized** to perform the following operations:
  - **Disassociate runtime environment** custom code execution environment
  - Making the **reproducible code** between development and production environments
  - **Insulate** each pipeline component (components can have their own version of the runtime environment, as well as different languages and libraries)
- **Continuous model delivery**: provides continuous **prediction services** to new models trained on new data thanks to the ML pipeline
- **Pipeline deployment** deployment of a **complete training pipeline** which runs **automatically** and **recurring** to deliver the trained model as a prediction service.

Level 1 ML pipeline automation

Level 1 often requires the installation of **certain prerequisites**

Feature Store: centralized repository where you **standardize** the definition, storage and access of features for training and inference.
It must provide an API for training and inference by batch or streaming (e.g: https://www.tecton.ai/)
Prevents training/inference discrepancies and facilitates data access.

Metadata management storage of executed pipeline and component versions, start and end dates and times, execution times, pipeline executor, pipeline arguments, code version, model performance metrics, etc.
Enables better **traceability**, **reproducibility**, **debug** and **comparison** results.

ML pipeline trigger training: allows you to restart training based on criteria such as the arrival of new data, performance degradation or significant changes in data distribution.

Level 1 ML pipeline automation

It is assumed that :

- deployment of new models **infrequent**
- you only manage **some ML pipelines**
- you deploy new models based **on new data** (rather than on new model ideas)

However, you must also **test new model ideas** and rapidly deploy new releases of ML components.

So if you manage **numerous ML pipelines in production** you need a **CI/CD configuration** to automate the creation, testing and deployment of ML pipelines!



https://ai-training.fr/

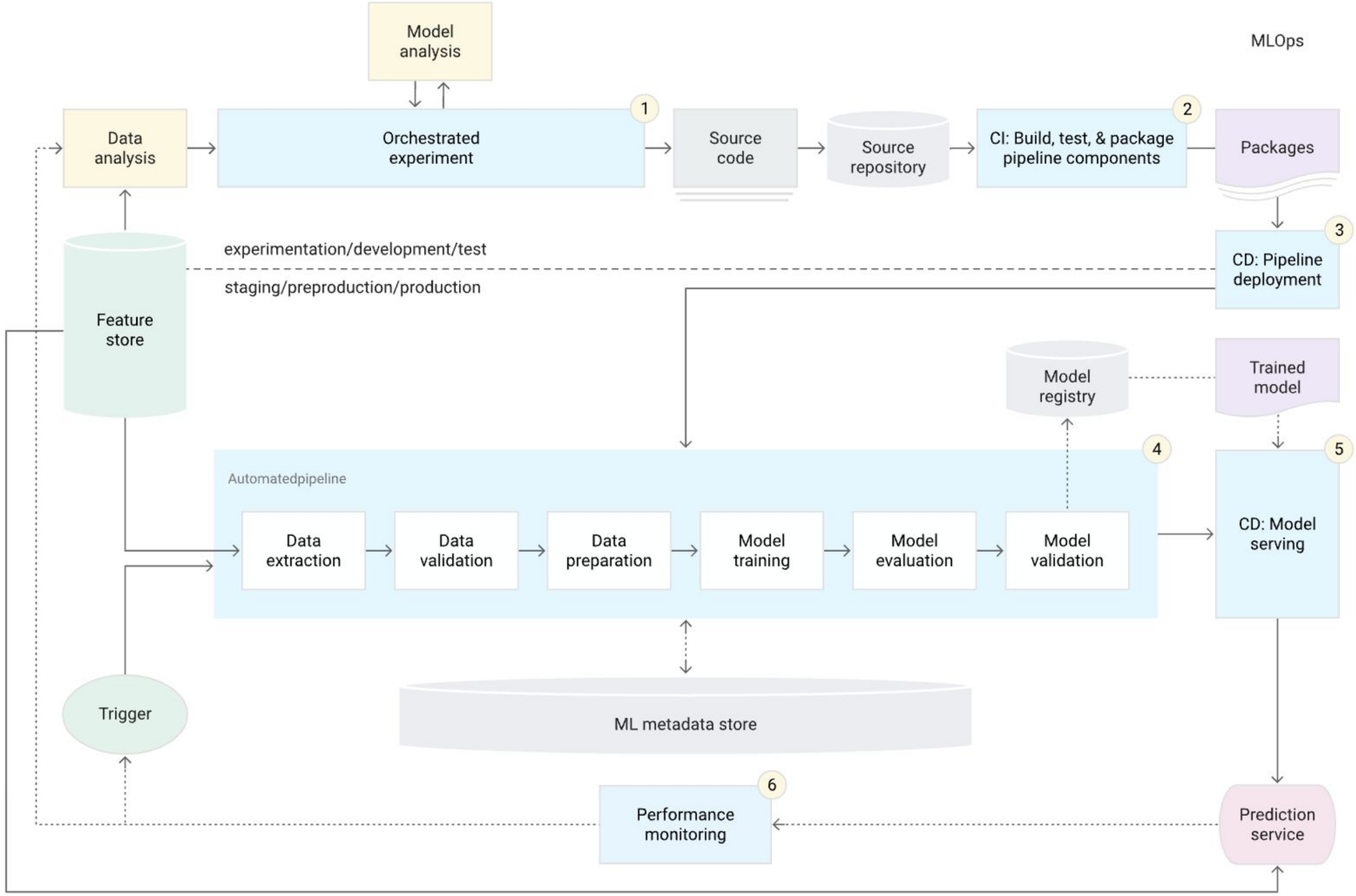
Level 2 CI/CD pipeline automation

Objectives

Quickly explore new ideas on feature extraction, model architecture and hyperparameters.
Implement these ideas, then **automatically create**, test and deploy the new pipeline components in the target environment.

How?

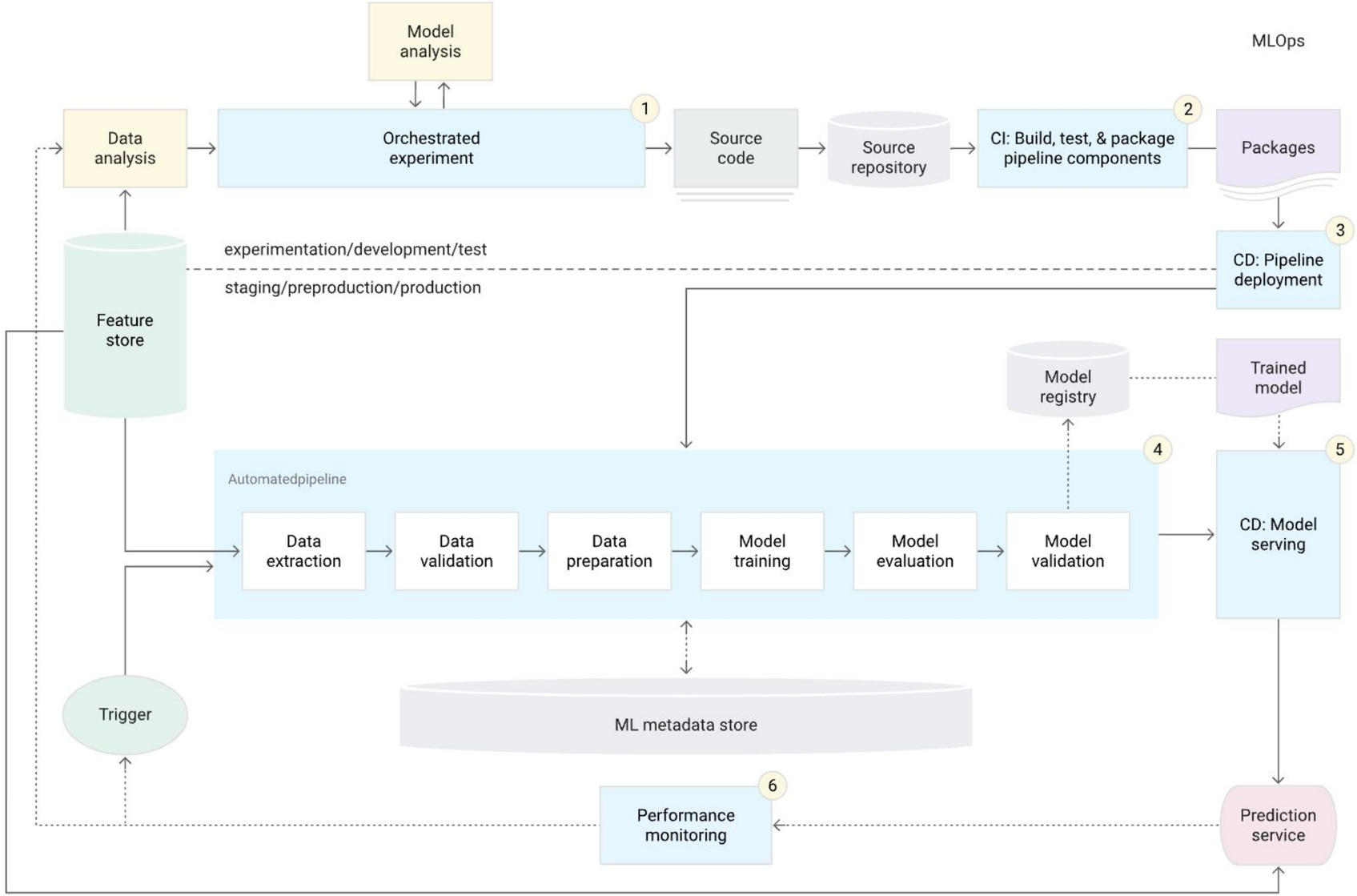
Implementation of the ML pipeline with integrated **CI/CD routines** automated



Level 2 CI/CD pipeline automation

This MLOps configuration includes the following components:

- Source control
- Testing and creating services
- Deployment services
- Model registry
- Feature Store
- ML metadata store
- ML Pipeline Orchestrator



Level 2 CI/CD pipeline automation

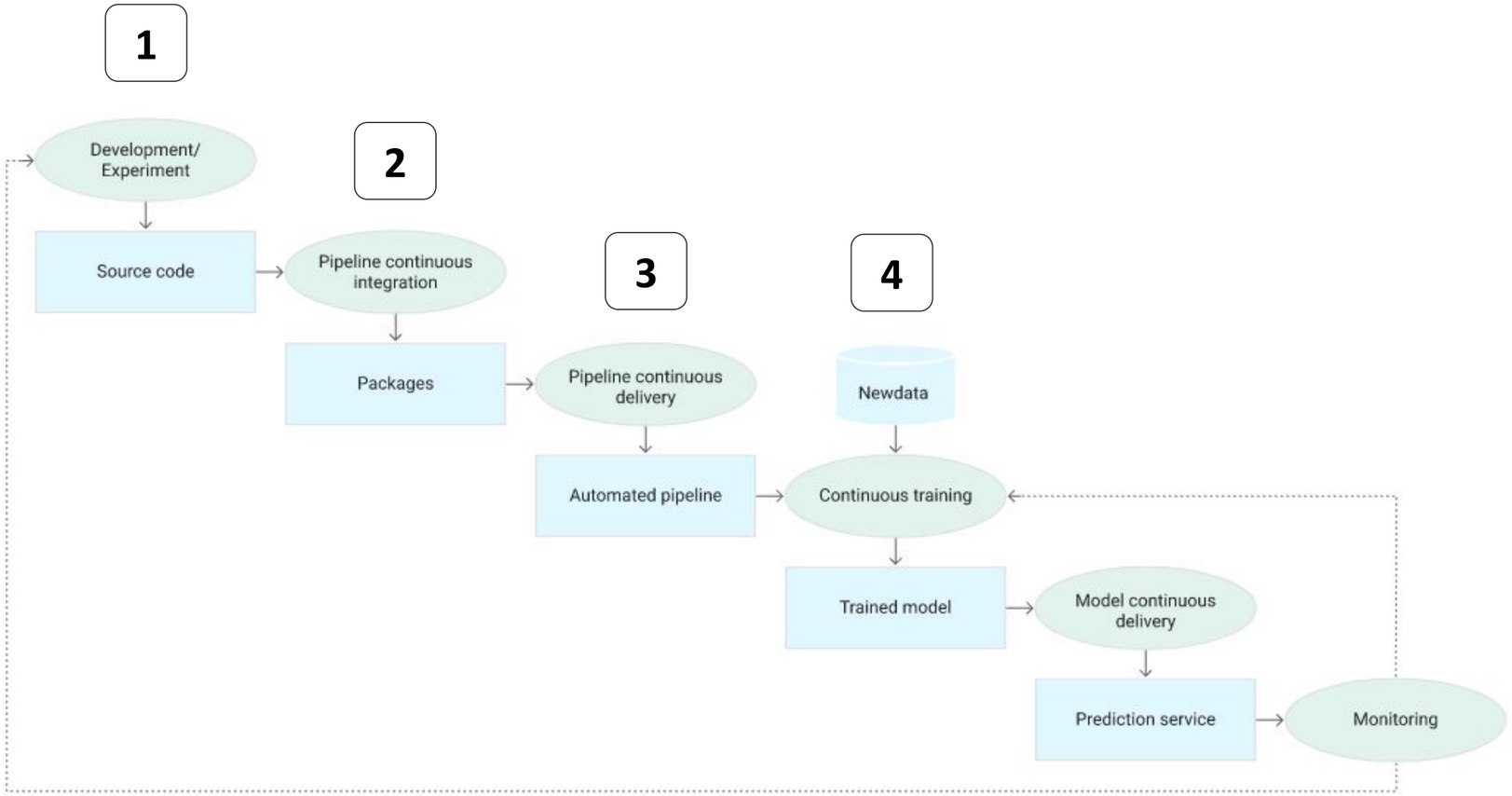
CI/CD routines

1. Development and experimentation orchestrate and iteratively test new ML algorithms. The source code then goes to a **source code repository** (Git).

2. **Pipeline CI**: build project with tests (packages, executables and artifacts)

3. **Pipeline CD** deployment of artifacts produced by the CI stage in the target environment

4. **Continuous training** (CT) the pipeline is automatically executed in production according to a program or in response to a request trigger (e.g. new data). The result of this step is a trained model that is transmitted to the model registry.

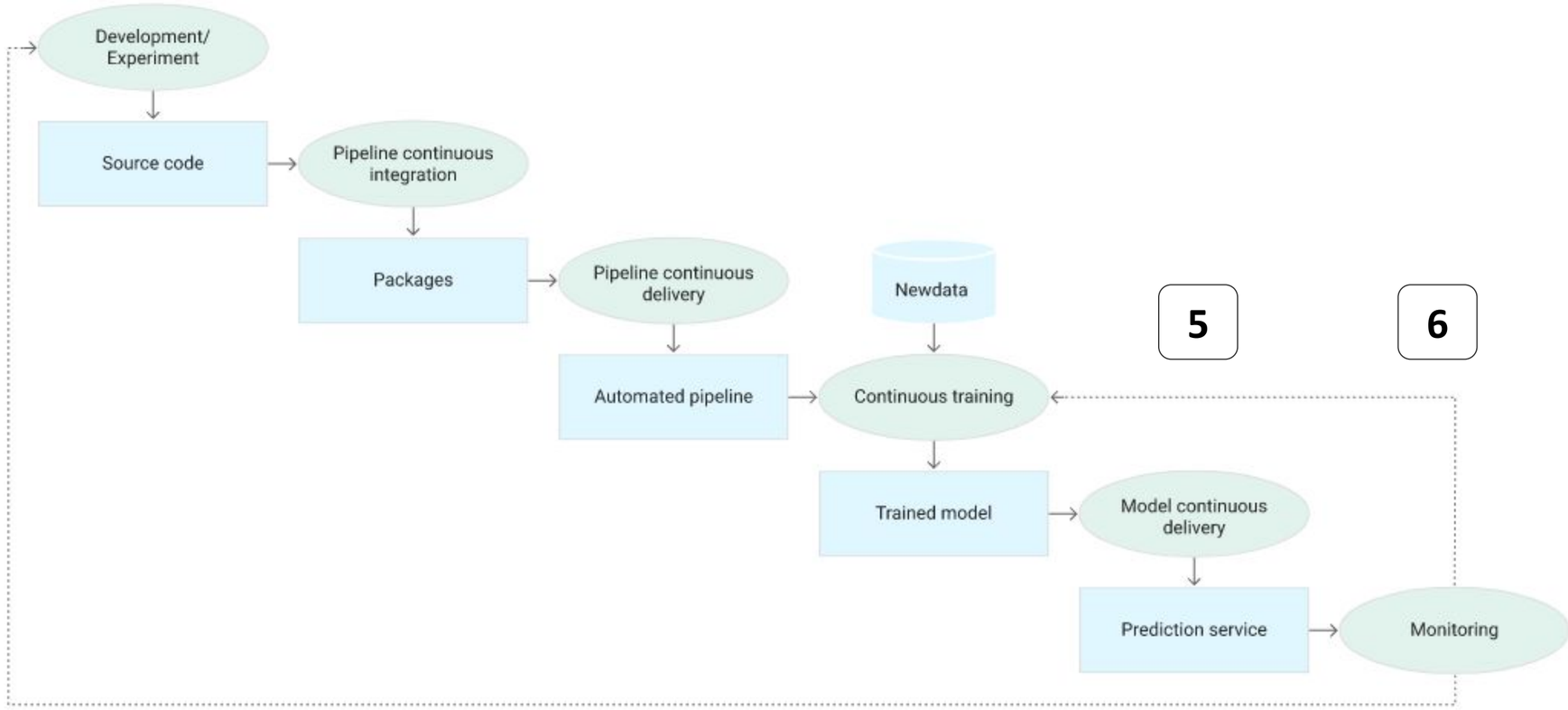


Level 2 CI/CD pipeline automation

CI/CD routines

5. Continuous model delivery the spread of the trained model as a service for predictions. The result of this step is a **deployed model prediction service**.

6. **Monitoring** the first step is to collect statistics on model performance based on online data. The result of this step is a **trigger** to run the pipeline or execute a **new test cycle**.



https://ai-training.fr/

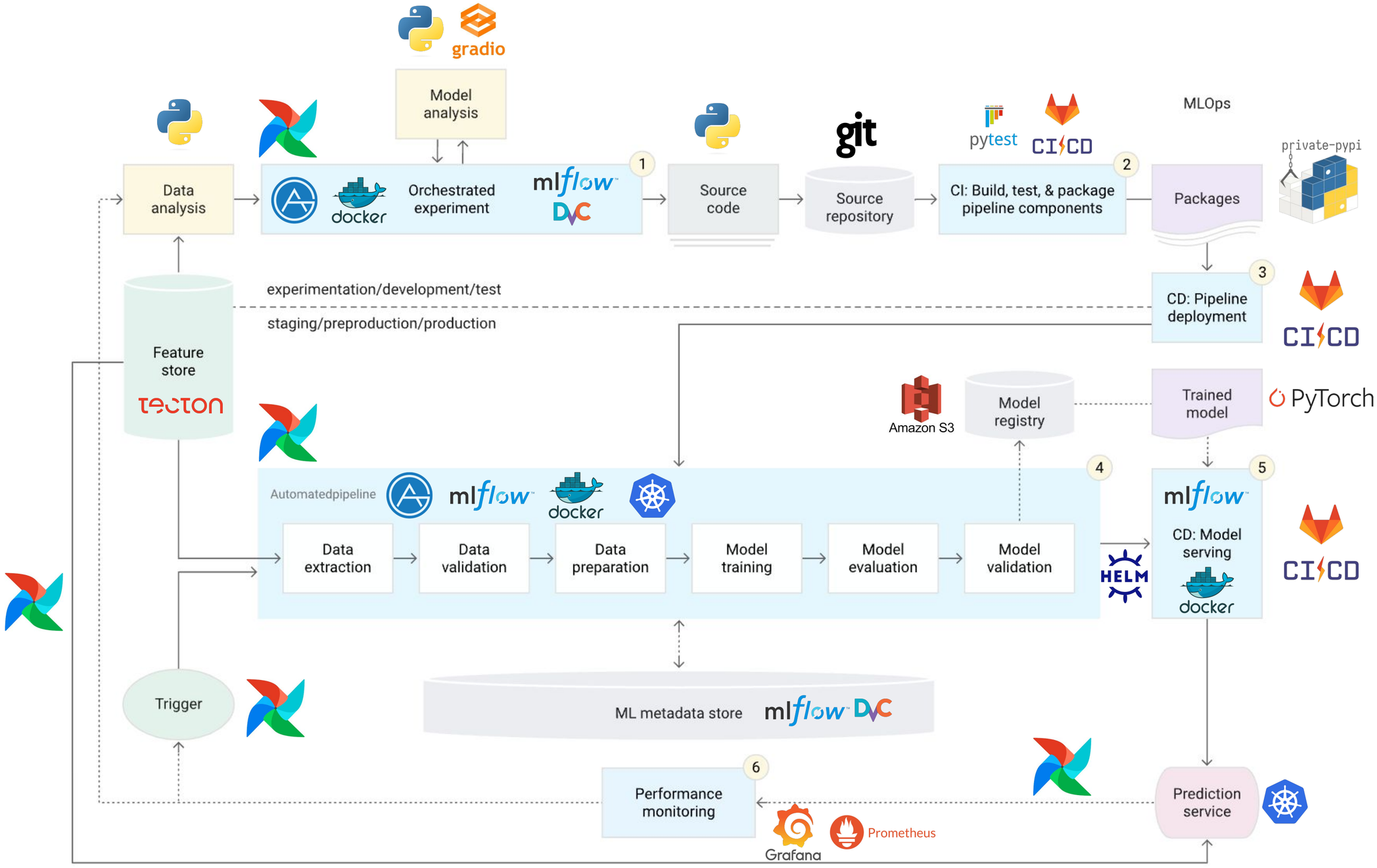
To sum up

Implementing ML in a production environment **isn't just** about deploying your model as an API for prediction.

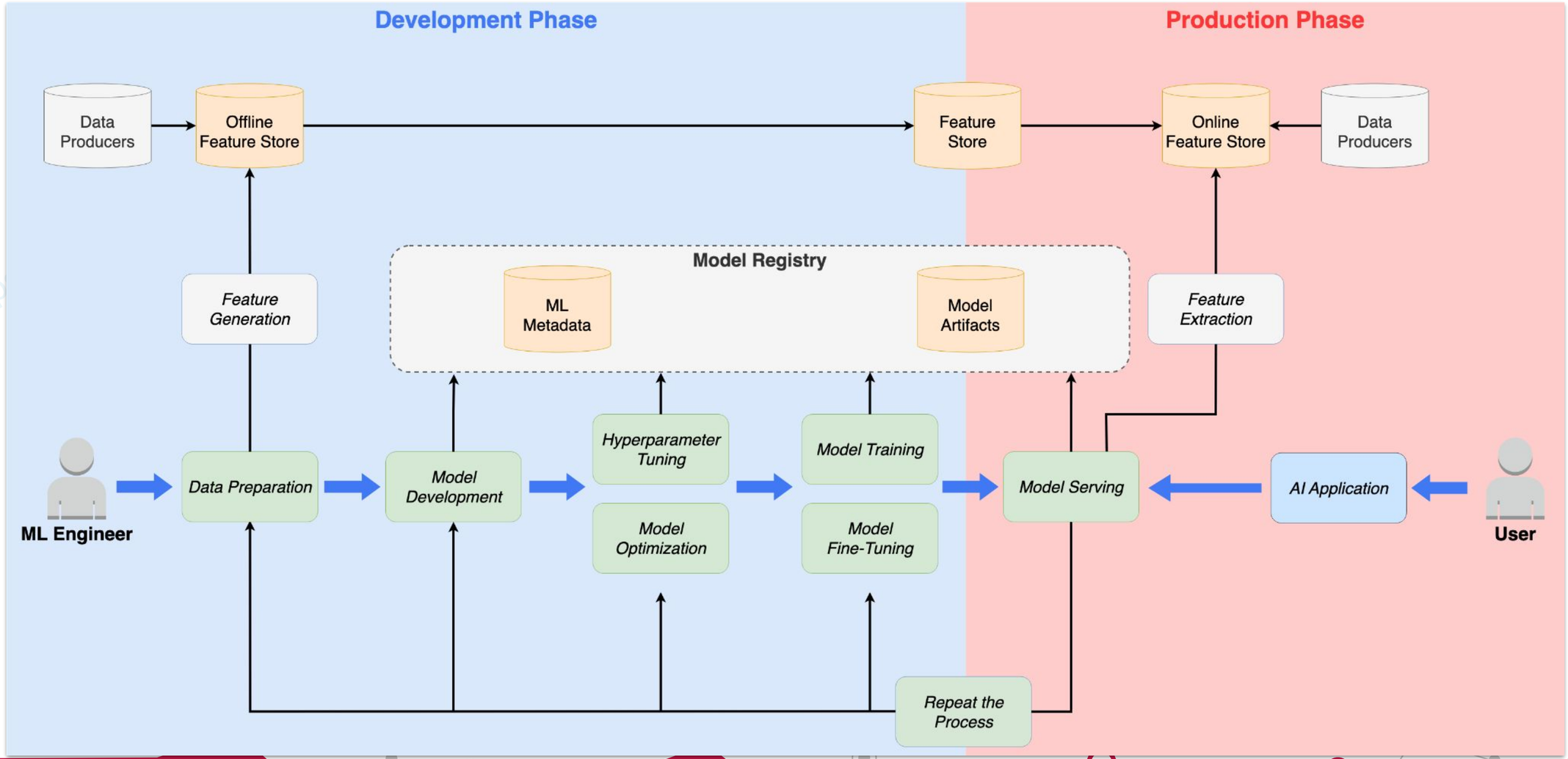
It also involves deploying an **ML pipeline** capable of **automating** the **retraining** and **deployment of new models**.
The configuration of **CI/CD** system allows you to automatically test and deploy new pipeline versions.

With such a system, you can cope with the rapid evolution of your data and your environment !

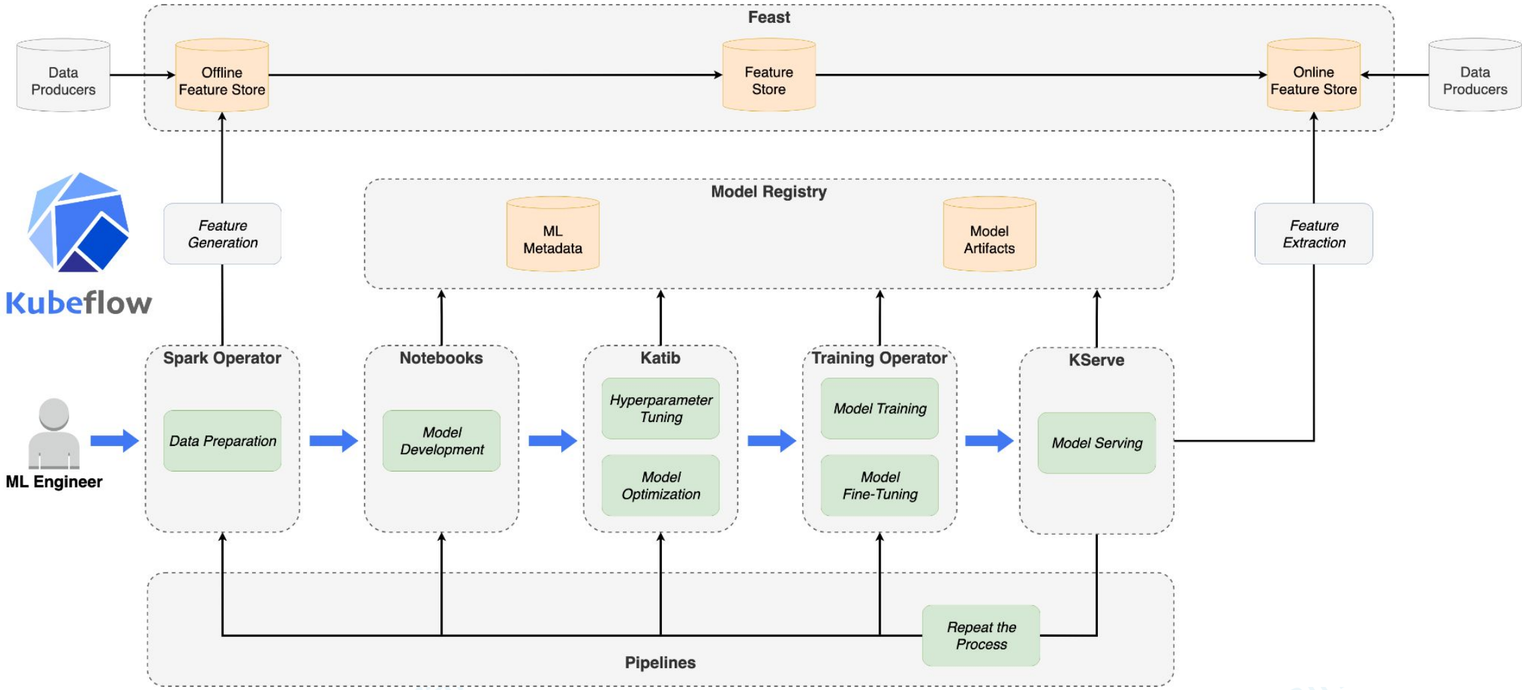
You can **implement gradually** these practices to improve the automation of your ML system development and its deployment in production.


gradio
Model analysis
MLOps
Data analysis
Orchestrated experiment
mlflow
DVC
docker
1
Source code
git
Source repository
pytest
CI/CD
CI: Build, test, & package pipeline components
2
Packages
private-pypi
3
CD: Pipeline deployment
CI/CD
experimentation/development/test
staging/preproduction/production
Feature store
tecton
Amazon S3
Model registry
Trained model
PyTorch
Automatedpipeline
4
Data extraction
Data validation
Data preparation
Model training
Model evaluation
Model validation
HELM
5
mlflow
CD: Model serving
docker
CI/CD
Trigger
ML metadata store
mlflow
DVC
6
Performance monitoring
Grafana
Prometheus
Prediction service

Adaptation of Kubeflow



Adaptation of Kubeflow



- Kubeflow Spark Operator: can be used for data preparation and feature engineering
- Kubeflow Notebooks: can be used for model development and experimentation
- Kubeflow Katib: can be used for model optimization and hyperparameter tuning using various AutoML algorithms
- Kubeflow Training Operator: can be used for large-scale distributed training or fine-tuning
- Kubeflow Model Registry: can be used to store ML metadata, model artifacts, and prepare models for the production department
- KServe: can be used for streaming inference and not batch inference via API
- Feast: can be used as a feature store
- Kubeflow Pipelines: can be used to build, deploy and manage every stage of the ML lifecycle

You can use most Kubeflow components standalone and integrate them into your existing AI/ML platform, or you can deploy the complete Kubeflow platform to get all Kubeflow components for an end-to-end ML lifecycle.

The levels of MLOps



The three levels of MLOps according to Google

0 manual process

1 pipeline ML

2 pipeline CI/CD

... and according to Microsoft

The 5 levels of MLOps

Microsoft defines 5 levels* of MLOps

- Level 0: no MLOps
- Level 1: devOps Non MLOps
- Level 2: automated drive
- Level 3: automated deployment
- Level 4: Fully automated re-training

https://ai-training.fr/

* Source :https://learn.microsoft.com/fr-fr/azure/architecture/example-scenario/mlops/mlops-maturity-model

Level 0 no MLOps

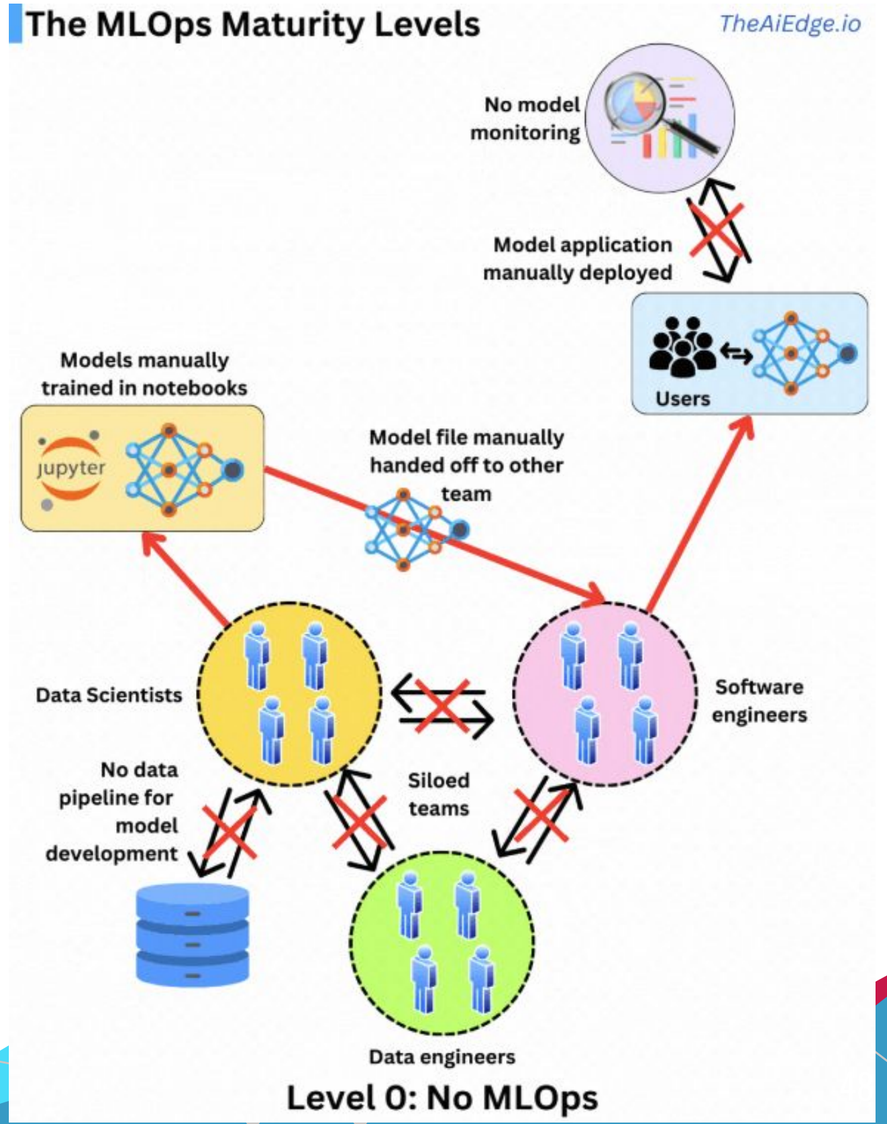
Level 0 is the **without automation**.

Generally speaking, Data Scientists are completely **disconnected** other teams.

This is the **guaranteed failure!**

It's possible to go through this stage to explore new ML opportunities, but if the company remains stuck at this stage, ML will contribute to the company's bottom line with great difficulty to the company's business.

People	Model creation	Putting the model into production	Application integration
• Isolated data scientists • Isolated data engineers (if any) • Isolated software engineers: reception **manual** model	• Collected data **manually** • The calculation is probably **not well managed** • Experiments are **not tracked** • The end result can be a unique model transmitted **manually** with inputs/outputs	• Procedure **manual** • The prediction script can be created manually after the experiments, without version control • Release managed by a single data scientist or data eng	• Highly dependent on data scientist expertise to implement the model • Production start-ups **manual** every time



Level 1 devOps Non MLOps

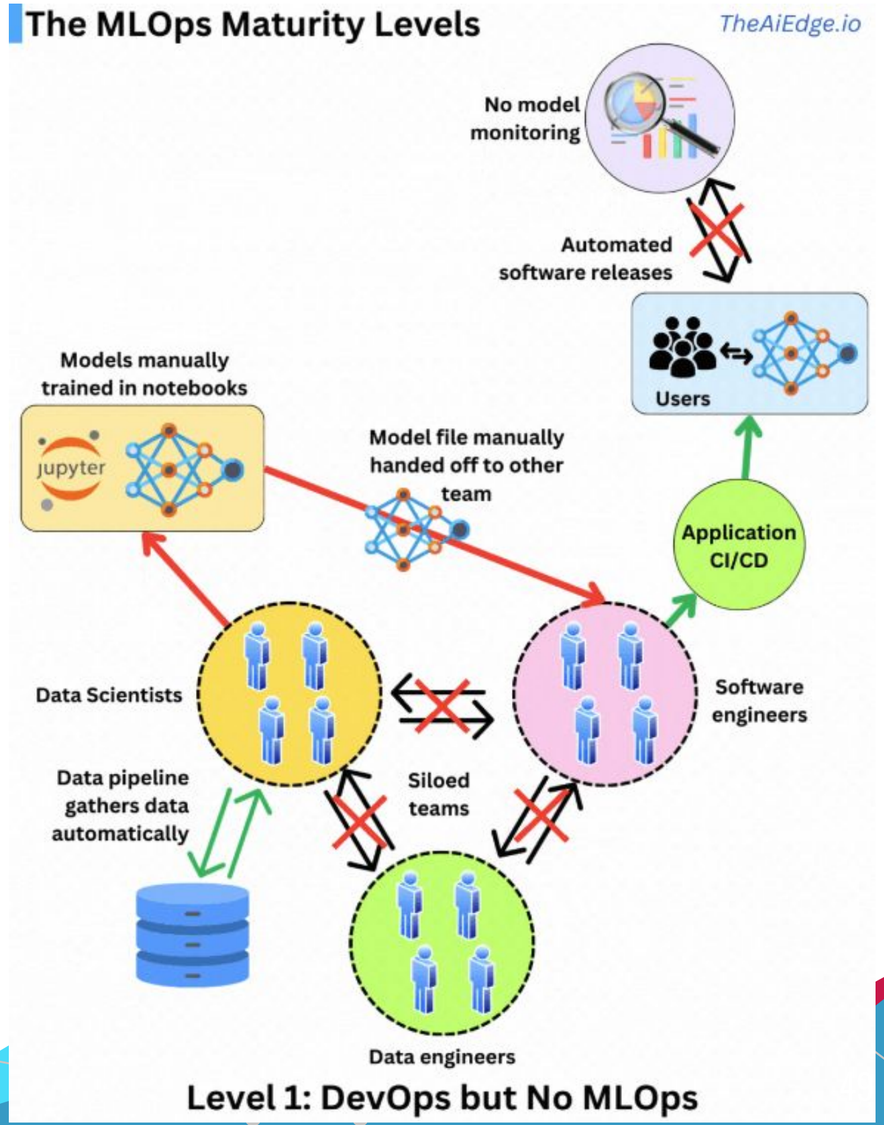
Level 1 corresponds to the point at which **ML are software applications**.

Visit **basic DevOps** principles are then applied at software level in production.

But there is an inability to realize the specificity of the ML operations, such as monitoring all stages in the construction of an ML system (integration, testing, release, deployment, infrastructure management, etc.).

On the development side, data pipelines are better established to streamline manual model development.

People	Model creation	Putting the model into production	Application integration
• Isolated data scientists • Data engineers (*if* exist) isolated • Isolated software engineers: manual model acceptance	• **Data pipeline** automatic data collection • Calculation **can be managed** or not • Experiments are not tracked • The end result can be a unique model transmitted **manually** with inputs/outputs	• Manual procedure • The prediction script can be created manually after the experiments, **probably with version control** • Trained model is transmitted **manually** to software engineers	• **Integration testing** are available for the model • Highly dependent on data scientist expertise to implement the model • Production start-ups **automated** • The application code contains **unit tests**



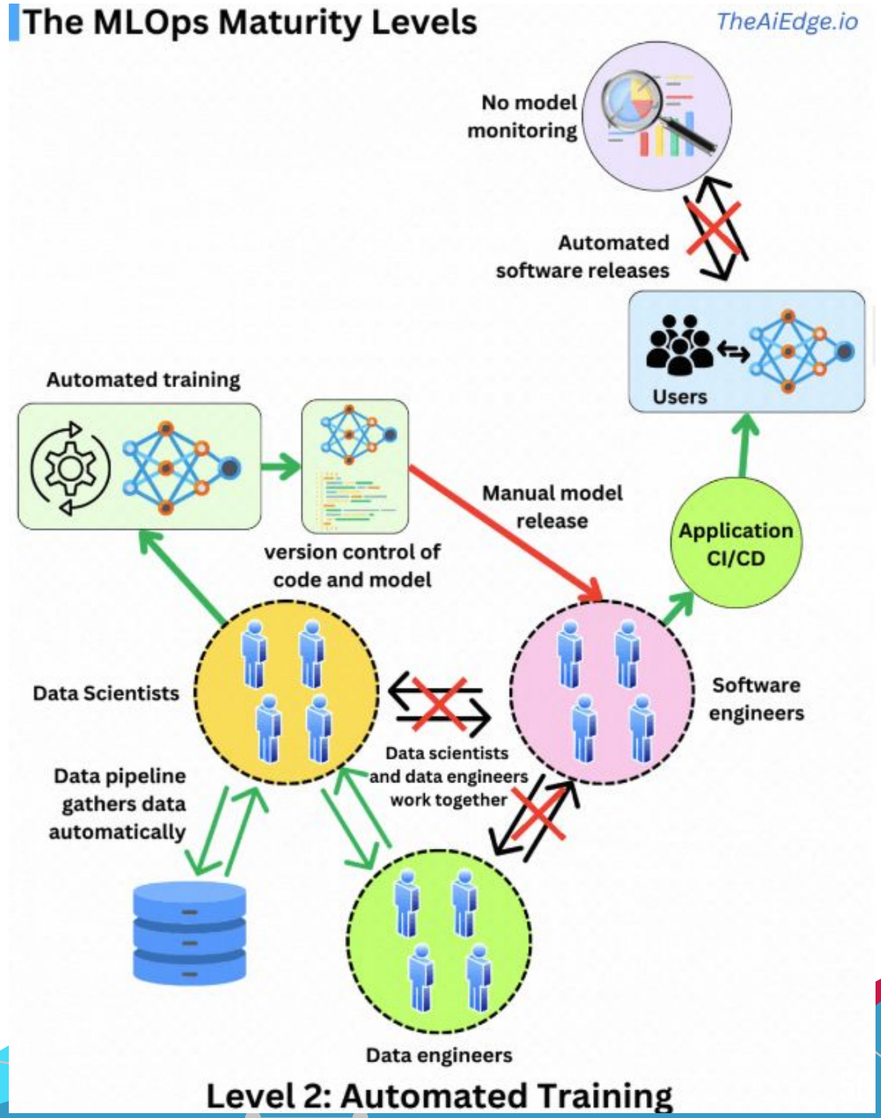
Level 2 automated drive

At level 2, things are starting to look interesting !

ML becomes sufficiently important for the company so that we can invest in the reduced development time models and errors.

Data teams **work more closely together** because **model development is automated** and **experiments are followed and reproducible**.

People	Model creation	Putting the model into production	Application integration
• Data Scientists work directly with the data eng to convert the experimental code into **reproducible scripts** • Data Engineers work with Data Scientists • Isolated software engineers: manual model acceptance	• The data pipeline collects data automatically • Calculation **managed** • **Tracking** experiment results • **Code** training and **models** are **versioned**	• **Manual** release • The prediction script**is versioned** with **tests** • Release **managed by team of software engineers**	• Basic integration tests exist for the • Highly dependent on the expertise of data scientists to implement the model • Application code contains unit tests



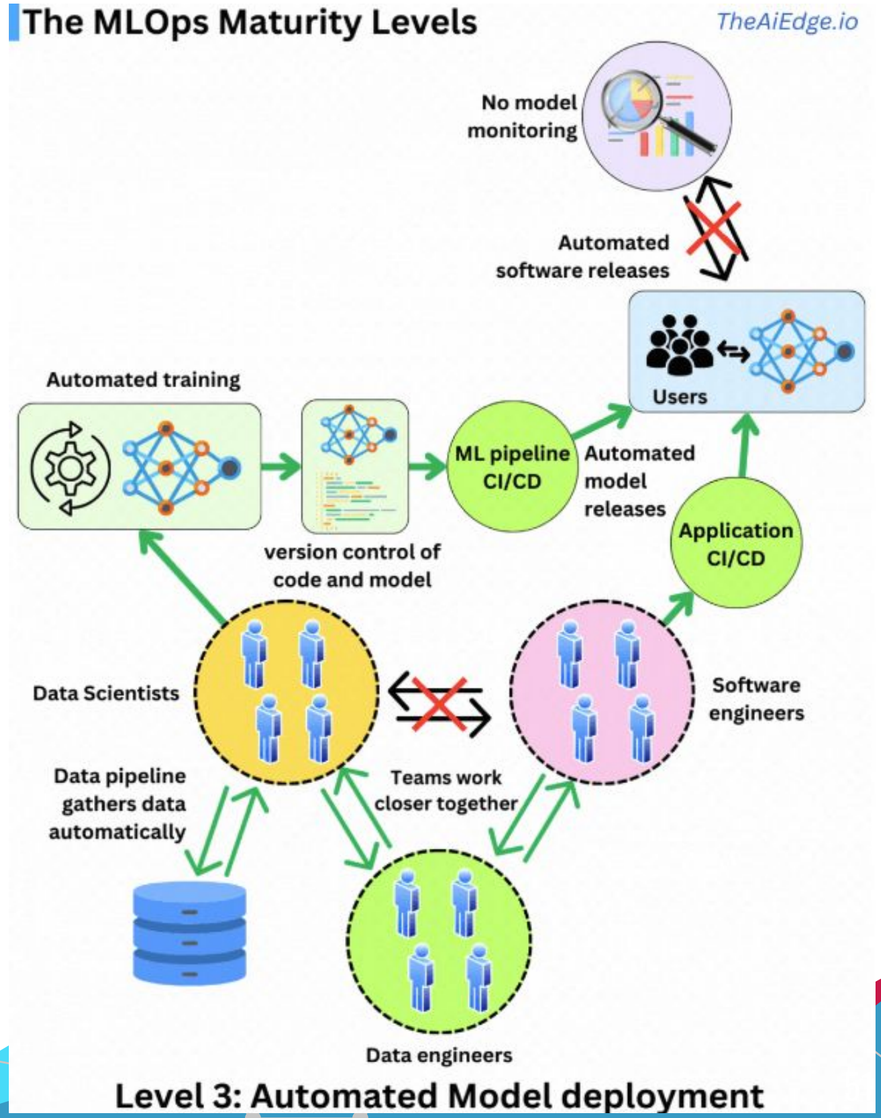
Level 3 automated deployment

If the ML is an important business issue for the company, **level 3 is the minimum bar to reach**.

This is where the transition from development to deployment is child's play.
DevOps principles extend to ML including **model and data testing**.

Models are **A/B tested** in production and **monitoring matures**. This enables rapid iteration and scaling of models for the ML engineering team.

People	Model creation	Putting the model into production	Application integration
• Data Scientist work directly with data eng to convert experimental code into reproducible scripts • Data Engineers work with Data Scientists and Software Engineers to **manage inputs/outputs** • Software Engineers work with data eng to **automate model integration**	• The data pipeline collects data automatically • Managed calculation • Tracking experiment results • Training code and resulting models are versioned	• **Automatic** release • The prediction script is versioned with tests • Release **managed by the pipeline** continuous delivery (CI/CD)	• **Tests** and integration for each model release • **Less dependent** the expertise of data scientists to implement the model • The application code contains **tests** unitary/integration

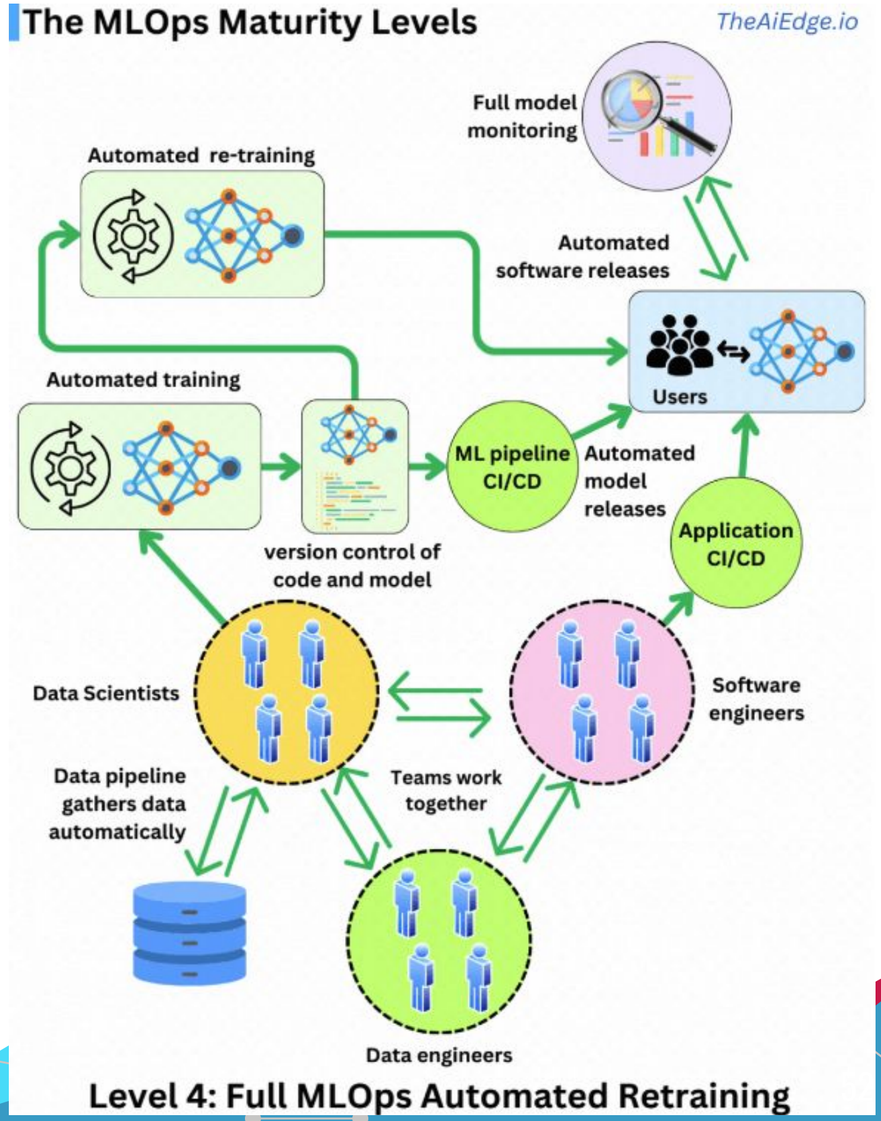


Level 4 fully automated retraining

Level 4 is the maturity level highest !

The teams **work together** training sessions recurring take place at least once a day and **everything is fully controlled**.

People	Model creation	Putting the model into production	Application integration
• Data Scientist work directly with data eng to convert experimental code into reproducible scripts • Data Engineers work with Data Scientists and Software Engineers to manage input/output • Software Engineers work with data eng to automate model integration and implement **metric** after deployment	• The data pipeline collects data automatically • **Automatically triggered re-training** based on criteria / production measurement • Managed calculation • Tracking experiment results • Training code and resulting models are versioned	• Automatic production startup • The prediction script is versioned with tests • Production launch managed by the Continuous Delivery Pipeline (CI/CD)	• Unit and integration testing for each model release • Less reliance on the expertise of data scientists to implement the model • Application code contains unit/integration tests

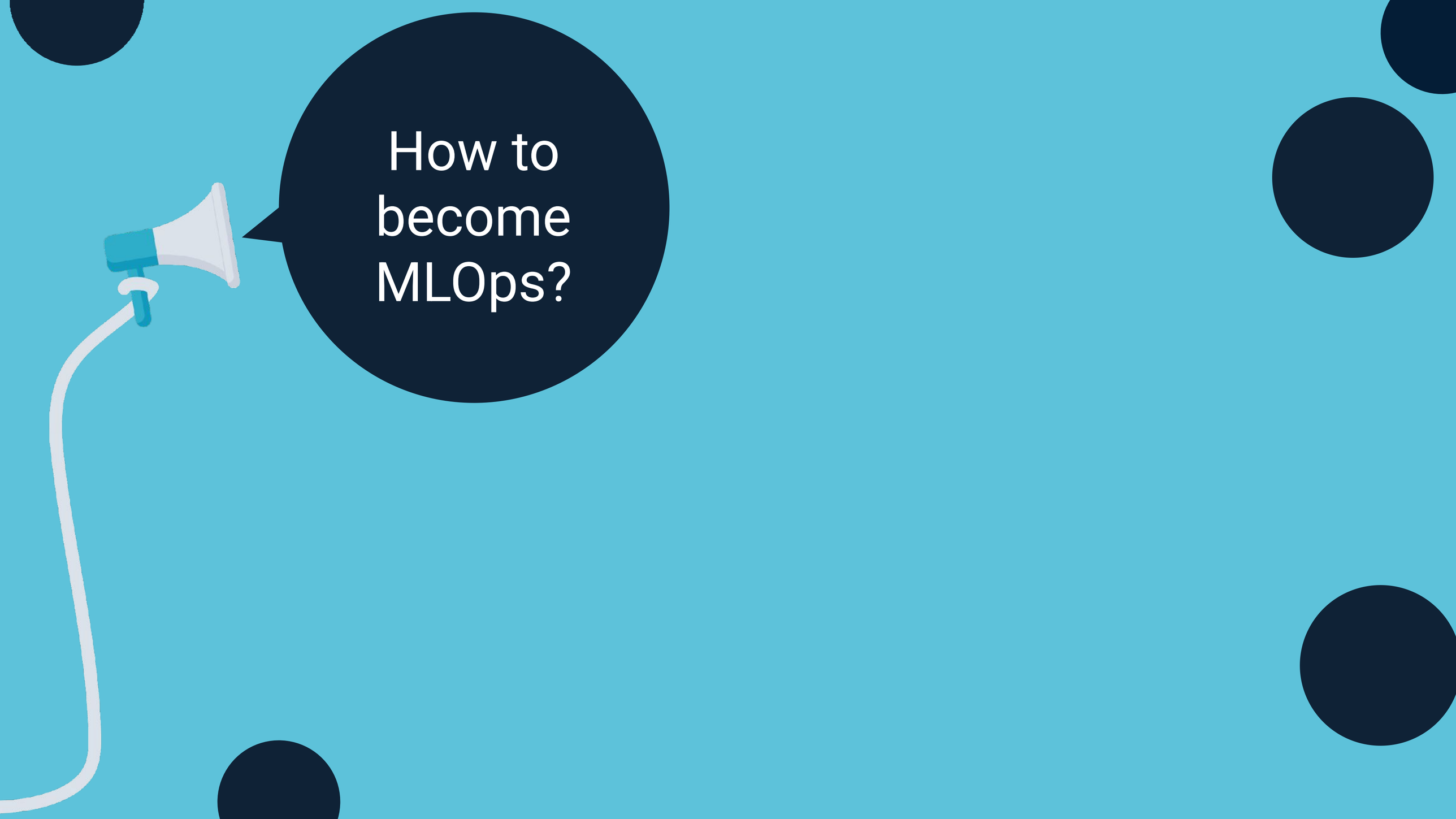




MLOPS

- What is MLOps?
- MLOps levels
- **How to become MLOps?**

https://ai-training.fr/


How to become MLOps?

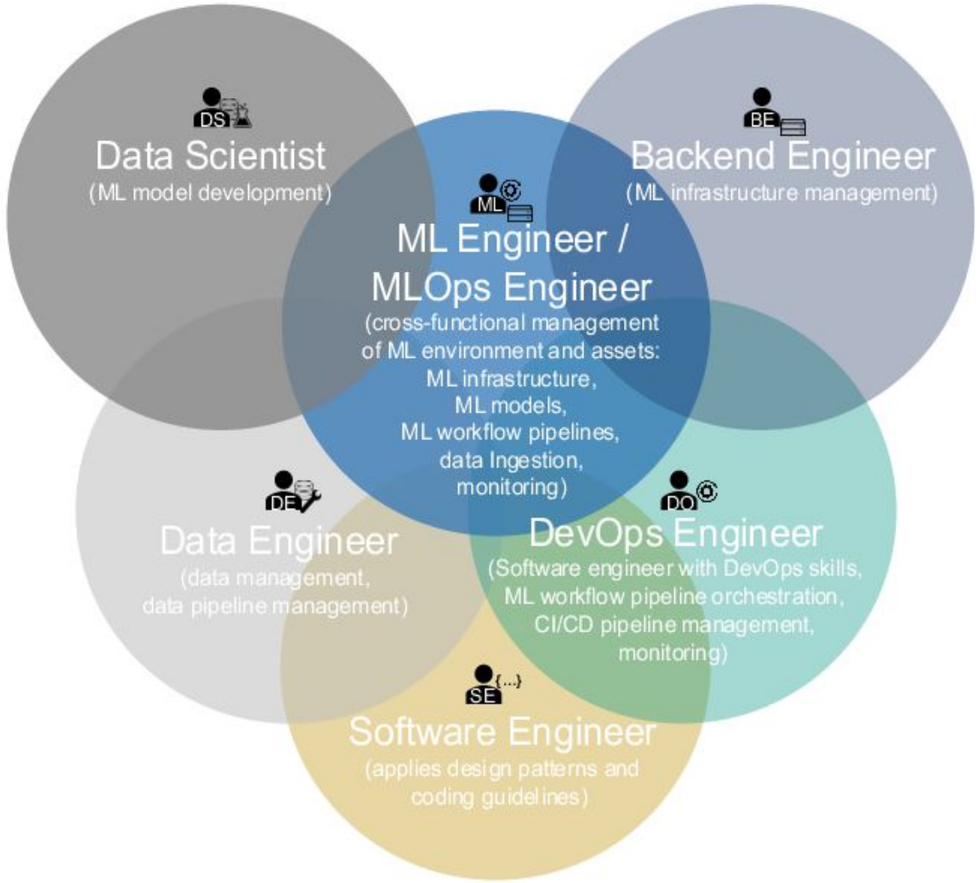
Definition of MLOps

Data Scientist: data analysis, building predictive models and extracting knowledge from data.

ML Engineer: to develop machine learning models with a focus on quality, scalability and robustness.

MLOps: responsible for the operationalization of machine learning models, deployment automation, lifecycle management and production monitoring.

Data Engineer: responsible for the design and maintenance of data infrastructures and pipelines for companies.



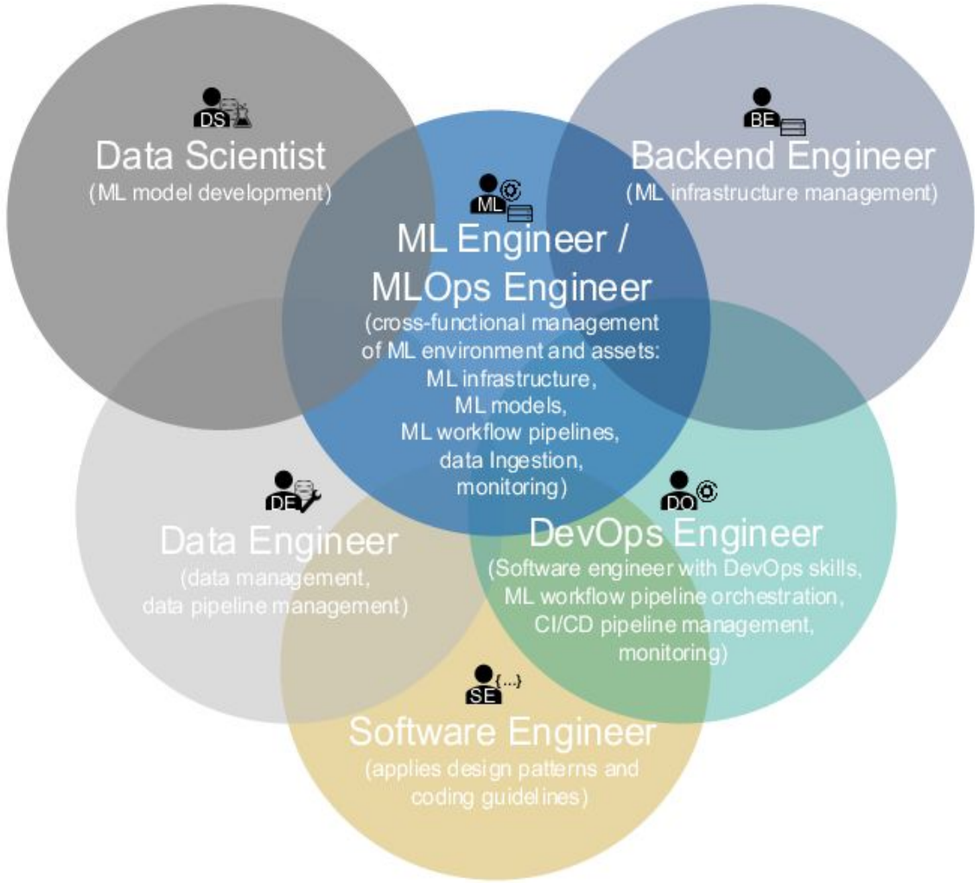
Source: https://arxiv.org/pdf/2205.02302.pdf

Definition of MLOps

DevOps: responsible for automating application development, testing, deployment and management.

Backend Engineer: responsible for building and maintaining server application systems, often in conjunction with the database.

Software Engineer: responsible for software development and maintenance in general.

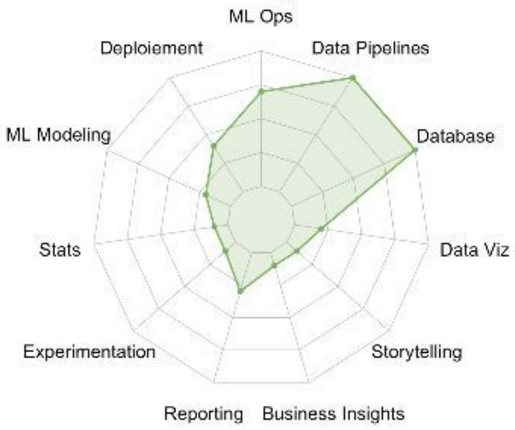


Source: https://arxiv.org/pdf/2205.02302.pdf

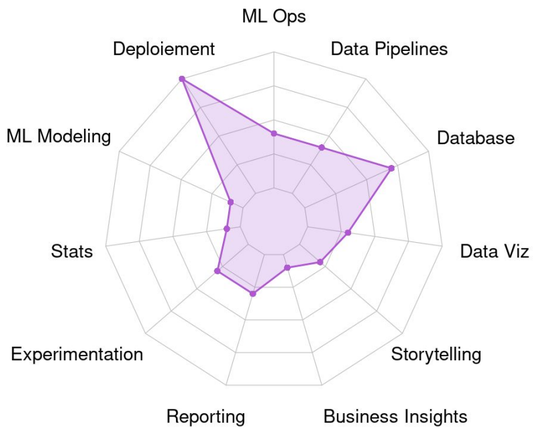
https://ai-training.fr/

Become a MLOps!

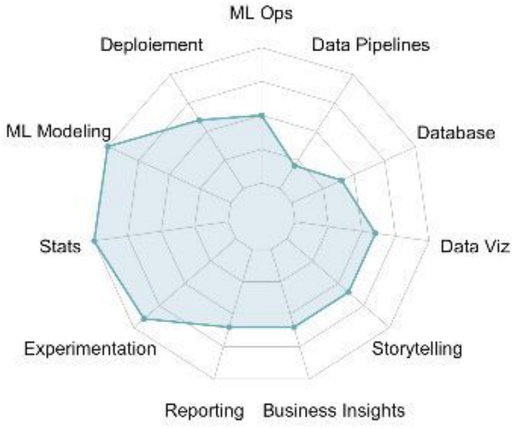
Data Engineer



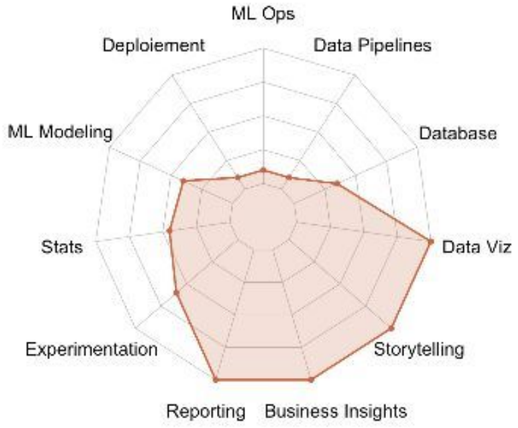
DevOps

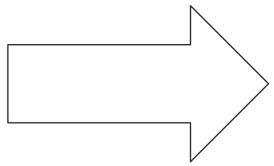


Data Scientist

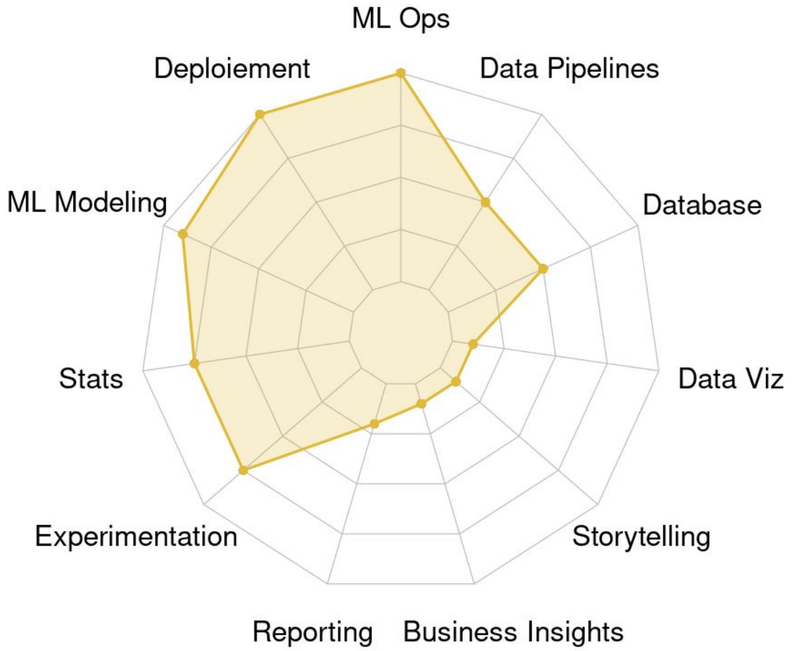


Data Analyst





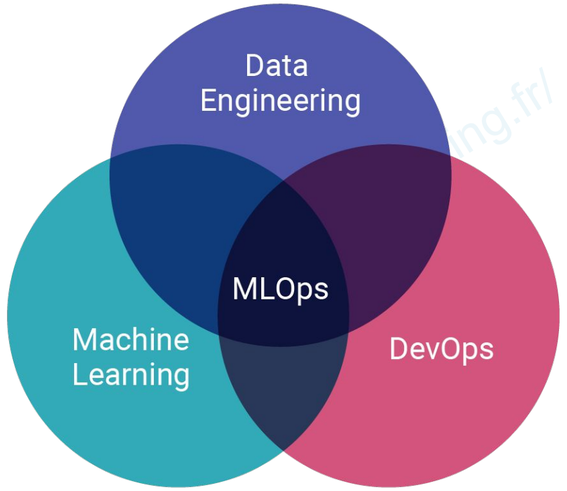
MLOps



Source :https://kevros.shinyapps.io/radar_skills/

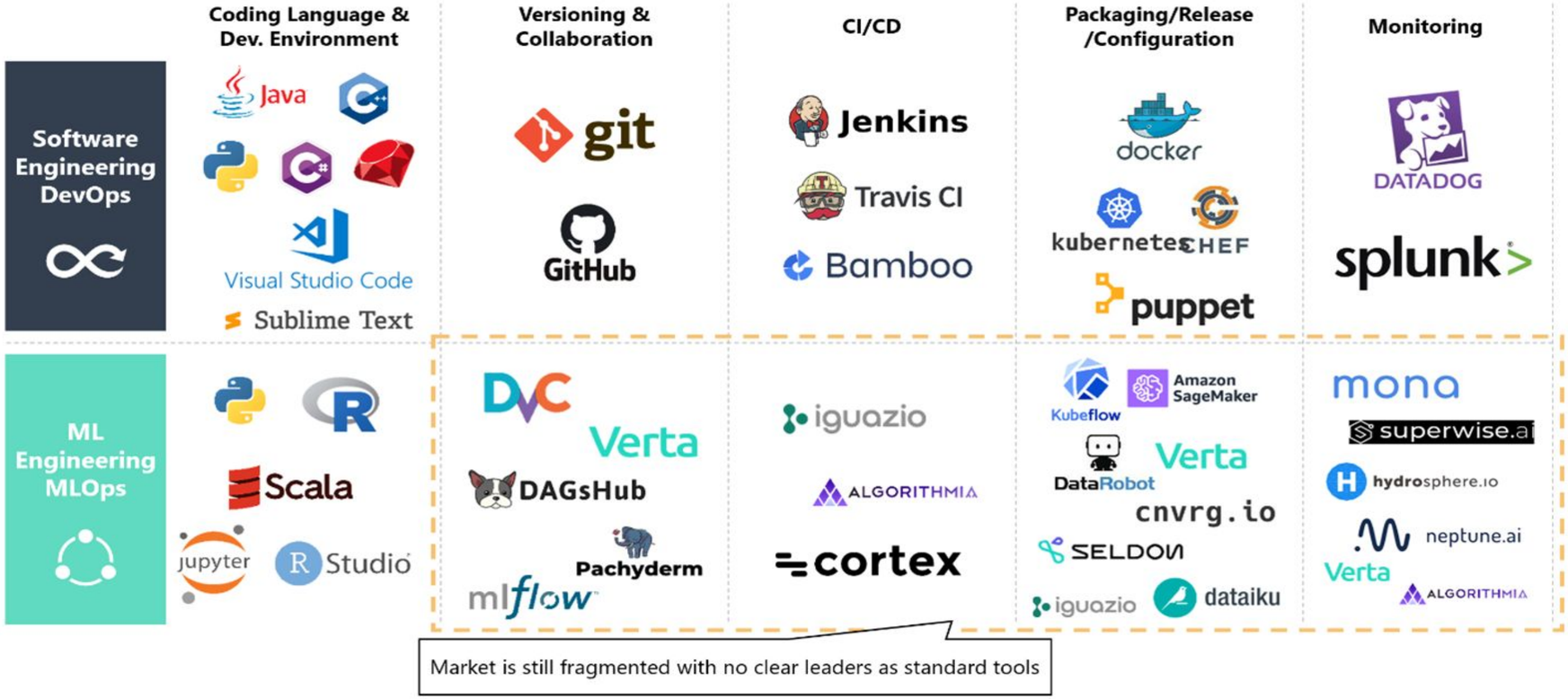
MLOps skills

- **Programming language** as a minimum **Python** and, if possible, **SQL**
- **Machine Learning and statistics concepts** and **AutoML**
- **Code versioning and CI/CD** such that **Git**
- **Automated test** ML models and code, such as **pytest** or **unittest**
- **Version management models** of ML, such as **MLflow, DVC** or **Kubeflow**
- **Serving as a model**, such as **TensorFlow Serving** or **FastAPI**
- **Deployment** ML models, such as **Docker** or **Kubernetes**
- **Management pipeline / ML workflow**, such as **Airflow** or **Prefect**
- **Monitoring** ML models, such as **Grafana** or **Kibana**
- **Creation of webapps to interact with models**, such as **Gradio** or **Streamlit**
- **Cloud solutions** such as **AWS SageMaker** or **Google cloud ML**
- **Technology watch** active
- **Flexibility, adaptability, communication and critical thinking skills**



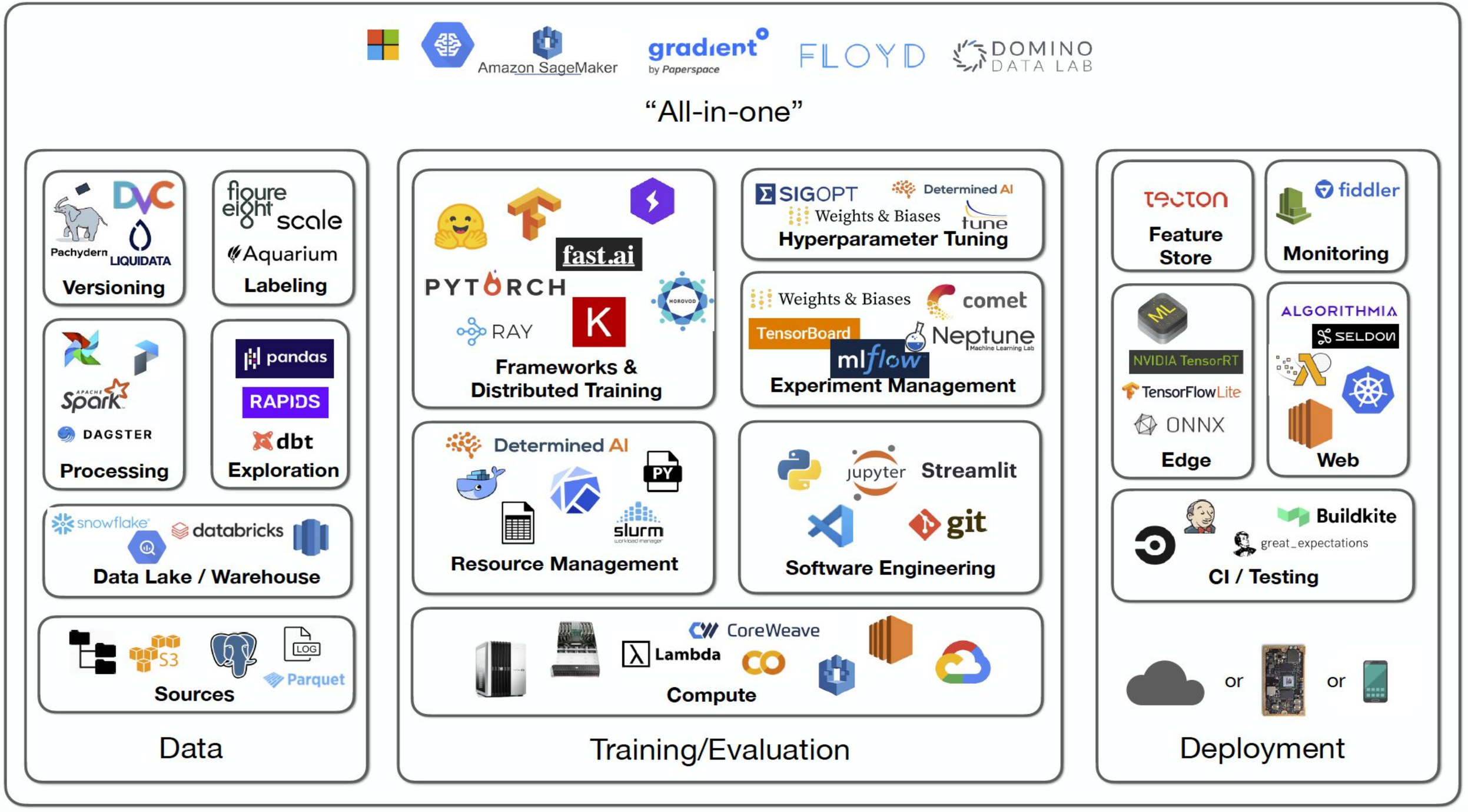
https://ai-training.fr/

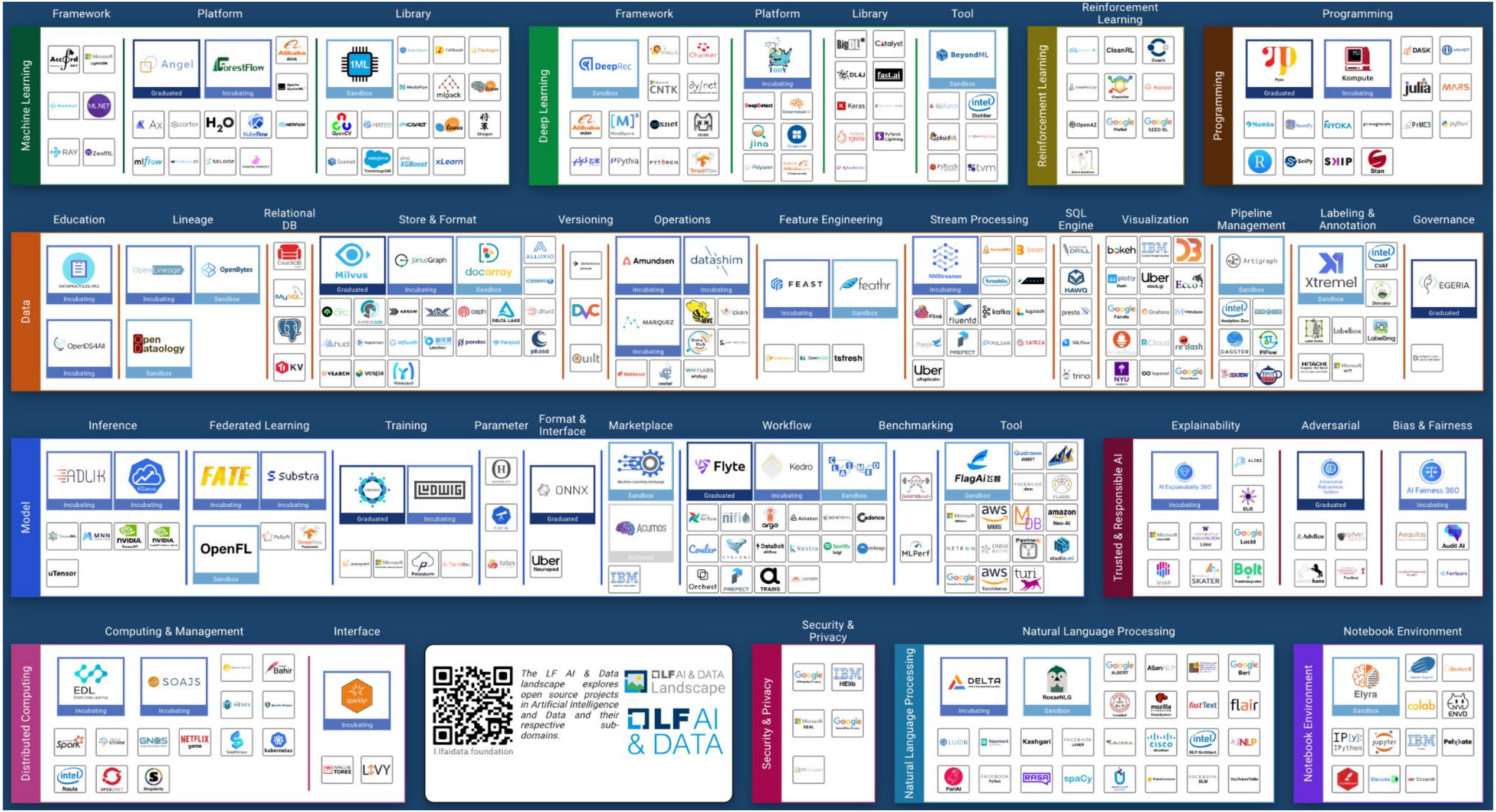
MLOps* tools



source :https://jumpcap.com/insights/mlops-the-key-to-scaling-machine-learning-in-an-enterprise

*Non-exhaustive list


Amazon SageMaker
gradient by Paperspace
FLOYD
DOMINO DATA LAB
"All-in-one"
Pachyderm
LIQUIDATA
Versioning
Aquarium
Labeling
Processing
pandas
RAPIDS
dbt
Exploration
snowflake
databricks
Data Lake / Warehouse
S3
Parquet
Sources
Data
fast.ai
PYTORCH
RAY
Frameworks & Distributed Training
SIGOPT
Determined AI
Weights & Biases
tune
Hyperparameter Tuning
Weights & Biases
comet
TensorBoard
Neptune
Machine Learning Lab
mlflow
Experiment Management
Determined AI
slurm
Resource Management
jupyter
Streamlit
git
Software Engineering
CoreWeave
Lambda
Compute
Training/Evaluation
tecton
Feature Store
fiddler
Monitoring
NVIDIA TensorRT
TensorFlowLite
ONNX
Edge
ALGORITHMIA
SELDON
Web
Buildkite
great_expectations
CI / Testing
or
or
Deployment


Machine Learning
Framework
Platform
Library
Deep Learning
Framework
Platform
Library
Tool
Reinforcement Learning
Reinforcement Learning
Programming
Programming
Data
Education
Lineage
Relational DB
Store & Format
Versioning
Operations
Feature Engineering
Stream Processing
SQL Engine
Visualization
Pipeline Management
Labeling & Annotation
Governance
Model
Inference
Federated Learning
Training
Parameter
Format & Interface
Marketplace
Workflow
Benchmarking
Tool
Trusted & Responsible AI
Explainability
Adversarial
Bias & Fairness
Distributed Computing
Computing & Management
Interface
Security & Privacy
Security & Privacy
Natural Language Processing
Natural Language Processing
Notebook Environment
Notebook Environment
The LF AI & Data landscape explores open source projects in Artificial Intelligence and Data and their respective sub-domains.
l.lfaidata.foundation
LF AI & DATA Landscape
LF AI & DATA

State-of-the-art MLOps technologies









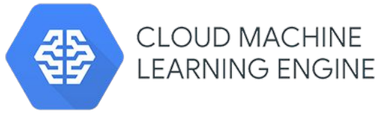
































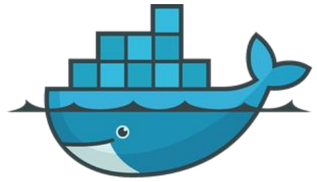


































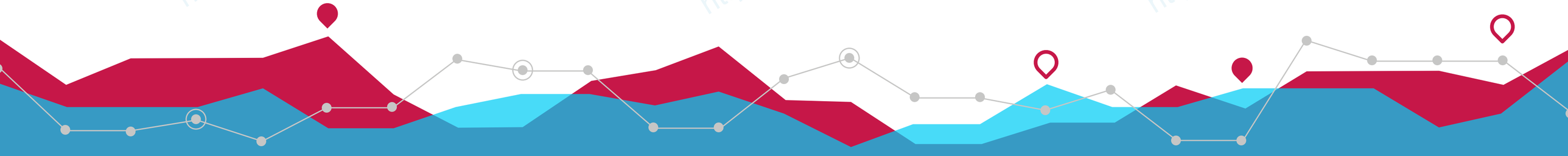


Best practices

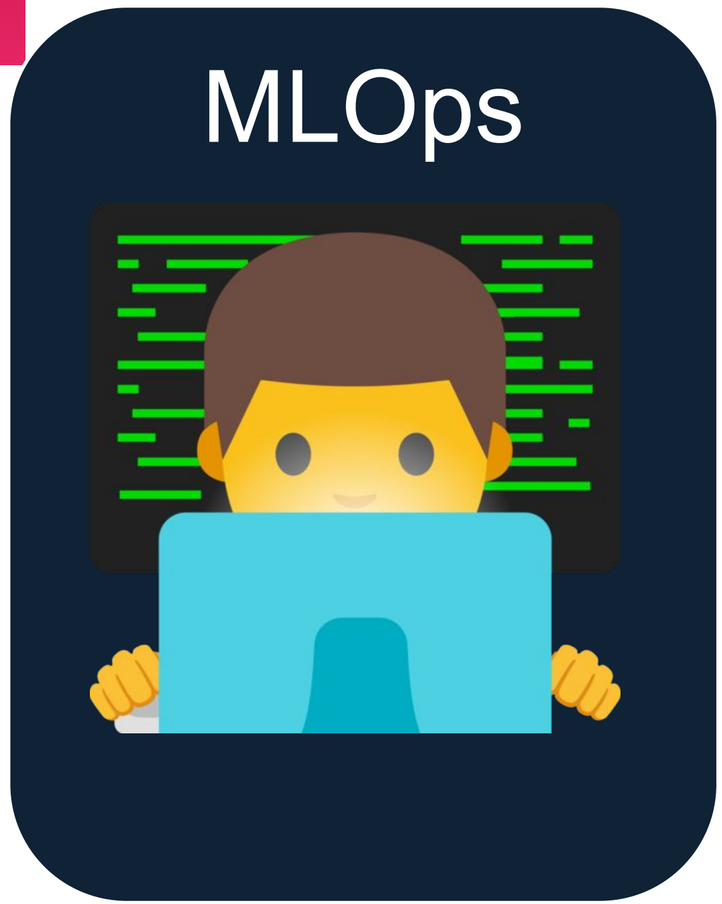
5 principles for improving success rates in ML project development and deployment:

- **Versatility**: cross-functional team responsible for the project from start to finish
- **Baseline**: implement a simple model (based on business / dummy rules) and test it from start to finish.
- **Iteration**: build better models and replace the simple model, check and repeat
- **Don't reinvent the wheel**: others have probably already created it better than we have!
- **Technology watch and critical thinking**: keep up to date and think about design/architecture

https://ai-training.fr/



Conclusion



Source: https://ml-ops.org/content/crisp-ml

- MLOps is a set of **skills** and **best practices** which requires close collaboration between your teams
- There are many **numerous tools** to help you implement MLOps in your company
- The real challenge is not to create an ML model with an API for prediction, but to create an ML model with an API for **prediction ML system integrated** and run it continuously in production!
- For a company to succeed in the ML field, teams must **work closely together** and aim for a **high level of automation** by eliminating the human element as much as possible as a source of error.
- MLOps is **crucial** for successful AI projects!

https://ai-training.fr/

Further information

- https://github.com/kelvins/awesome-mlops
- https://ml-ops.org/
- https://arxiv.org/pdf/2202.10169.pdf
- https://services.google.com/fh/files/misc/practitioners_guide_to_mlops_whitepaper.pdf

- AWS :https://aws.amazon.com/fr/sagemaker/mlops/
- Azure :https://learn.microsoft.com/fr-fr/azure/architecture/reference-architectures/ai/mlops-python
- Google :https://cloud.google.com/architecture/setting-up-an-mlops-environment?hl=fr

https://ai-training.fr/